



Katedra Oprogramowania

Wydział Informatyki

Informatyka

Data 14-06-2026

Rok II Semestr IV Zespół: Mateusz Tręda Karol Kuc Dominik Jarzyło Adres repozytorium: https://github.com/Domssson/IO-Volleyball	Prowadzący: dr inż. Marcin Czajkowski
--	--

Podział pracy:

- Mateusz Tręda: 33%
- Karol Kuc: 33%
- Dominik Jarzyło: 33%

1.) Zadanie

Projekt będzie systemem wspomagającym pracę oraz działanie kadry sportowej w analizie meczów.

System pozwala na rejestrację, prowadzenie oraz analizę meczy siatkarskich oraz osób w mecz zaangażowane. Zanim mecz się rozpocznie to trzeba zarezerwować halę na termin rozgrywek, ustalić drużyny i przydzielić sędziego. Koordynator rezerwuje termin wydarzenia, wynajmuje sprzęt i przydziela drużyny oraz ich trenerów. Całą rejestracją, weryfikacją, zapisem danych zajmuje się Administrator.

2.) Cel budowy systemu

Główne cele systemu:

- **Zastąpienie manualnego wprowadzania danych:** wprowadzenie uczenia maszynowego, brak konieczności ręcznego przekazywania danych systemowi.
- **Dostarczanie analizy w czasie rzeczywistym:** skrócenie czasu od zakończenia akcji do uzyskania informacji zwrotnej do minimum.
- **Zaawansowana analiza:** wykorzystanie algorytmów Computer Vision do mierzenia parametrów niemierzalnych dla ludzkiego oka.

Zakres systemu:

- **Przetwarzanie obrazu (Computer Vision):** Wykrywanie zawodników, piłki oraz linii boiska w strumieniu wideo (live lub plik).
- **Moduł klasyfikacji zagrań:** Automatyczne rozpoznawanie typów akcji (zagrywka, przyjęcie, wystawa, atak, blok, obrona) oraz ich efektu.
- **Panel analityczny (Dashboard):** Generowanie dynamicznych raportów: heatmap ataku, rozkład rozegrania, heatmap zagrywki.
- **Baza danych i Archiwum:** Przechowywanie historycznych danych o zawodnikach i meczach w celu analizy trendów sezonowych.
- **Eksport danych:** Generowanie plików kompatybilnych z formatami ligowymi (np. arkusze PDF/XLS).

Kontekst systemu:

- **Źródła danych:** Kamery IP (RTMP/RTSP), kamery stacjonarne, pliki wideo lokalne.
- **Użytkownicy:** Trenerzy (podgląd taktyczny), statystycy (weryfikacja danych AI), zawodnicy (analiza własnych błędów).
- **Otoczenie:** System może integrować się z zewnętrznymi bazami danych ligowych lub systemami transmisyjnymi (nakładki graficzne TV).

Korzyści z wdrożenia:

a) Mierzalne (Ilościowe)

1. **Redukcja czasu pracy:** Skrócenie czasu potrzebnego na przygotowanie pełnego raportu z meczu o 70-80% (z kilku godzin do kilkunastu minut).
2. **Zmniejszenie kosztów:** Brak konieczności zatrudniania wysokiej klasy specjalisty-statystyka w mniejszych klubach (system „obsługuje się sam”).
3. **Wzrost precyzji danych:** Wyeliminowanie błędów ludzkich (tzw. "human error") wynikających ze zmęczenia statystyka podczas długich setów.
4. **Parametry fizyczne:** Możliwość dokładnego pomiaru np. zasięgu w ataku z dokładnością do 1-2 cm.

b) Niemierzalne (Jakościowe)

1. **Lepsza jakość treningu:** Trenerzy mogą natychmiast korygować błędy ustawienia zawodników na podstawie heatmap.
2. **Przewaga psychologiczna:** Możliwość błyskawicznego wykrycia powtarzalnych schematów przeciwnika w trakcie trwania seta (tzw. *Real-time tactical adjustment*).
3. **Nowoczesny wizerunek klubu:** Budowanie profesjonalnej kultury opartej na danych (*Data-Driven Culture*).
4. **Łatwiejsza edukacja zawodników:** Wizualizacja błędów (overlay na wideo) jest znacznie bardziej przekonująca dla zawodnika niż sucha statystyka w tabeli.

3.) Słownik

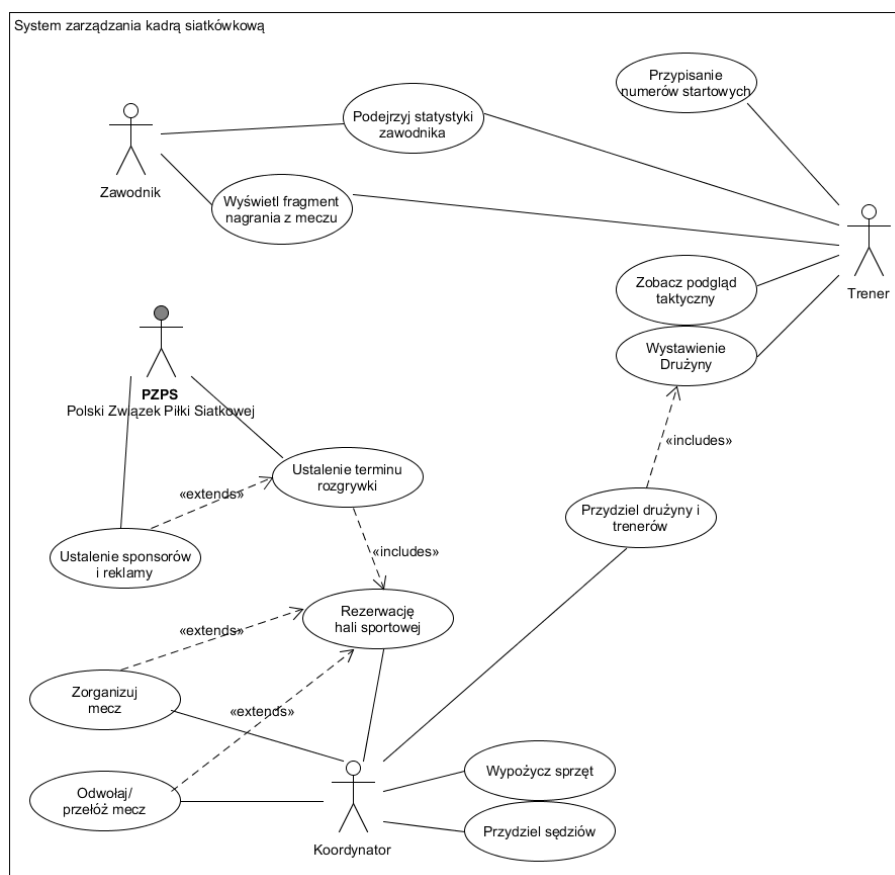
Słownik definiujący ważne, specyficzne terminy związane z dziedziną problemu, wykorzystywane w projekcie systemu analizy meczów siatkarskich.

- **System analizy meczów** – oprogramowanie wspomagające analizę przebiegu meczu siatkarskiego poprzez automatyczne przetwarzanie obrazu, identyfikację zdarzeń oraz generowanie statystyk.
- **Computer Vision (widzenie komputerowe)** – dziedzina informatyki zajmująca się analizą obrazu i wideo, wykorzystywana w systemie do wykrywania zawodników, piłki oraz elementów boiska.
- **Uczenie maszynowe (Machine Learning)** – metody pozwalające systemowi uczyć się na podstawie danych, stosowane do klasyfikacji zagrań i analizy wzorców gry.
- **Strumień wideo (Video Stream)** – ciągły przesył danych wideo z kamery (np. RTSP/RTMP), będący wejściem dla systemu analizy.
- **Detekcja obiektów (Object Detection)** – proces identyfikacji i lokalizacji obiektów (np. zawodników, piłki) w obrazie.
- **Śledzenie obiektów (Object Tracking)** – proces monitorowania ruchu wykrytych obiektów w kolejnych klatkach wideo.
- **Klasyfikacja zagrań** – automatyczne przypisywanie typu akcji (np. zagrywka, atak, blok) na podstawie analizy obrazu.
- **Akcja (Rally Event)** – pojedyncze zdarzenie w trakcie wymiany (np. atak, przyjęcie, blok).
- **Rally (wymiana)** – sekwencja akcji od momentu zagrywki do zdobycia punktu przez jedną z drużyn.
- **Heatmapa (mapa cieplna)** – wizualna reprezentacja intensywności zdarzeń (np. ataków) na boisku.
- **Dashboard (panel analityczny)** – interfejs użytkownika prezentujący dane statystyczne, wizualizacje oraz przebieg meczu w czasie rzeczywistym.
- **Analiza w czasie rzeczywistym (Real-time Analysis)** – przetwarzanie danych i prezentowanie wyników bez zauważalnego opóźnienia.
- **Overlay (nakładka wizualna)** – graficzne elementy nanoszone na obraz wideo (np. trajektoria piłki, pozycje zawodników).

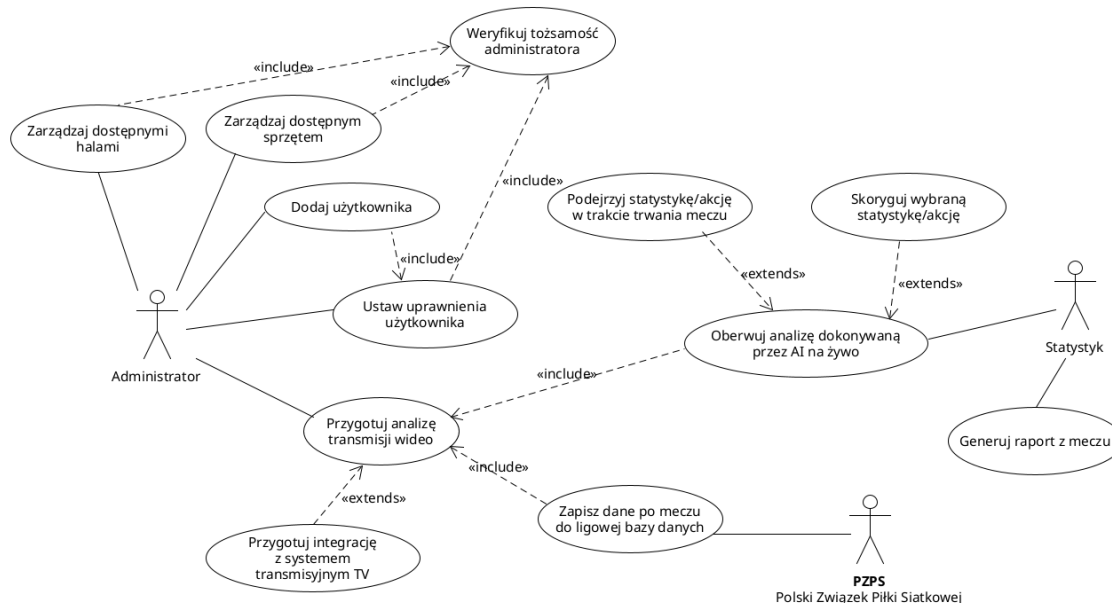
- **Model AI** – wytrenowany algorytm odpowiedzialny za analizę obrazu i klasyfikację zdarzeń.
- **Metryki meczu** – wskaźniki statystyczne opisujące przebieg gry (np. skuteczność ataku, liczba błędów, liczba bloków).
- **Skuteczność ataku (Attack Efficiency)** – procentowy stosunek udanych ataków do wszystkich prób.
- **Błąd (Error)** – akcja zakończona niepowodzeniem, skutkująca punktem dla przeciwnika wynikająca z nieprawidłowego zagrania zawodnika (atak w aut, wpadnięcie w siatkę, przejście linii itp.).
- **As (Ace)** – zagrywka bezpośrednio kończąca akcję zdobyciem punktu.
- **Blok (Block)** – akcja obronna przy siatce mająca na celu zatrzymanie ataku przeciwnika.
- **Obrona (Dig)** – skuteczne podbicie piłki po ataku przeciwnika.
- **Przyjęcie (Reception)** – pierwsze odbicie piłki po zagrywce przeciwnika.
- **Przyjęcie perfekcyjne (Perfect Reception)** – przyjęcie po którym zawodnik rozgrywający dysponuje każdą opcją, przyjęcie między 0,5- 1m od siatki
- **Wystawa (Set)** – podanie piłki przez rozgrywającego do zawodnika atakującego.
- **Rotacja zawodników** – aktualne ustawienie zawodników na boisku wynikające z zasad gry.
- **Koordinator** – osoba odpowiedzialna za organizację meczu (rezerwacja hali, przydział drużyn i sędziów).
- **Administrator** – użytkownik systemu odpowiedzialny za zarządzanie danymi, użytkownikami i konfiguracją systemu.
- **Statystyk** – osoba nadzorująca poprawność danych generowanych przez system.
- **Trener** – użytkownik korzystający z danych analitycznych do podejmowania decyzji taktycznych.
- **Archiwum danych** – baza przechowująca historyczne dane o meczach i zawodnikach.
- **Eksport danych** – proces generowania raportów w formatach zewnętrznych (np. PDF, XLS).

4.) Perspektywa przypadków użycia

Diagramy przypadków użycia



Rysunek 1 Diagram przypadków użycia dla organizacji meczu



Rysunek 2 Diagram przypadków użycia dla przebiegu użycia programu w kontekście meczowym

Opisy aktorów

Termin	Definicja
Trener	Osoba posiadająca licencję trenerską, uprawniona do prowadzenia drużyny siatkówki na określonym poziomie rozgrywek a także wglądu w nagrania wideo z meczów.
Zawodnik	Osoba będąca członkiem drużyny siatkówkowej. Korzysta z systemu w celach informacyjnych – przegląda własne statystyki, ogląda fragmenty nagrań z meczów oraz śledzi wyniki rozgrywek. Najczęściej korzysta z systemu po zakończeniu meczu lub treningu.
Koordinator	Pracownik operacyjny PZPS lub wyznaczona osoba odpowiedzialna za bieżącą logistykę rozgrywek. Zarządza organizacją konkretnych meczów: rezerwuje hale sportowe, przydziela sędziów, wypożycza sprzęt sportowy oraz obsługuje sytuacje wyjątkowe, takie jak odwoływanie lub przekładanie meczów.
Statystyk	Osoba zajmująca się wyliczaniem statystyk w trakcie trwania meczu. Nadzoruje i w razie potrzeby koryguje pracę systemu analizy AI. Na koniec sporządza raport z meczu.
Administrator	Osoba uprawniona do zarządzania obiektami sportowymi i sprzętem, dodawania nowych użytkowników. Przygotowuje również system w celu rozpoczęcia analizy meczu.
PZPS	Organizacja zarządzająca rozgrywkami. Inicjuje i nadzoruje kluczowe procesy: publikowanie wyników, ustalanie terminów rozgrywek, przydzielanie drużyn i trenerów oraz pozyskiwanie sponsorów. Działa zarówno przez pracowników etatowych, jak i deleguje zadania koordinatorom.

Opisy tekstowe przypadków użycia

UC-01: Podejrzyj statystyki zawodnika

Aktorzy: Zawodnik, Trener

Ciąg podstawowy: Użytkownik (Zawodnik lub Trener) loguje się do systemu i wybiera opcję przeglądania statystyk. Wskazuje konkretnego zawodnika (lub siebie). System pobiera z bazy danych statystyki meczowe (punkty, asysty, zagrywki, bloki, przyjęcia) i wyświetla je w formie tabelarycznej lub graficznej. Użytkownik może filtrować dane według sezonu, meczu lub kategorii statystyk.

Ciąg alternatywny / awaryjny: Jeśli zawodnik nie posiada jeszcze żadnych rozegranych meczów w systemie, wyświetlany jest komunikat „Brak danych statystycznych”. Jeśli sesja użytkownika wygaśa, system przekierowuje do ekranu logowania.

Częstotliwość: Kilka–kilkanaście razy dziennie na zawodnika. Czas typowy: poniżej 3 sekund. Czas maksymalny: 10 sekund (przy dużym obciążeniu serwera).

Wartość dla aktorów: Zawodnik uzyskuje wgląd w swój własny rozwój sportowy i historię występów. Trener zyskuje narzędzie analityczne do oceny formy zawodników i podejmowania decyzji taktycznych.

UC-02: Wyświetl fragment nagrania z meczu

Aktorzy: Zawodnik, Trener (inicjatorzy)

Ciąg podstawowy: Użytkownik (Zawodnik lub Trener) wybiera mecz z listy rozegranych spotkań. System wyświetla dostępne fragmenty wideo powiązane z danym meczem. Użytkownik wybiera interesujący fragment (np. konkretny set, akcja, zagrywka). System odtwarza nagranie w odtwarzaczu wbudowanym.

Ciąg alternatywny / awaryjny: Jeśli nagranie nie zostało jeszcze przetworzone lub przesłane, system informuje: „Nagranie niedostępne – oczekiwanie na przetworzenie”. Jeśli połączenie jest zbyt wolne, system proponuje niższą jakość odtwarzania.

Częstotliwość: Kilka razy tygodniowo; spiętrzenia w trakcie przygotowań do kolejnego meczu (2–3 dni przed spotkaniem). Czas typowy: uruchomienie odtwarzacza w ciągu 5 sekund. Czas maksymalny: 20 sekund przy dużym pliku wideo.

Wartość dla aktorów: Zawodnik może analizować własne błędy i sukcesy. Trener wykorzystuje nagrania do przygotowania taktycznego zespołu i analizy gry przeciwnika.

UC-03: Ustalenie terminu rozgrywki

Aktorzy: PZPS (inicjator), Koordynator (wykonawca); powiązane «includes»: Rezerwację hali sportowej; «extends»: Ustalenie sponsorów i reklamy

Ciąg podstawowy: PZPS inicjuje proces planowania terminarza sezonu lub konkretnej kolejki rozgrywkowej. Koordynator w systemie przegląda dostępność hal sportowych i drużyn. Wybiera datę, godzinę i miejsce meczu, sprawdza konflikty terminowe. Po zatwierdzeniu system tworzy wydarzenie meczowe, automatycznie wywołując rezerwację hali (UC: Rezerwację hali sportowej) i organizację meczu (UC: Zorganizuj mecz).

Ciąg alternatywny / awaryjny: Jeśli żądana hala jest niedostępna w danym terminie, system sugeruje alternatywne obiekty lub terminy. Jeśli drużyna zgłosiła konflikt terminowy (turniej, zgrupowanie reprezentacji), system flaguje kolizję i wymaga potwierdzenia lub zmiany terminu.

Częstotliwość: Masowo na początku sezonu (planowanie całego terminarza – kilkaset meczów), a następnie kilka razy w miesiącu przy zmianach. Czas typowy: 10–15 minut na zaplanowanie jednej kolejki. Czas maksymalny: 2 godziny przy planowaniu pełnego terminarza sezonu.

Wartość dla aktorów: PZPS uzyskuje spójny, zatwierdzony terminarz rozgrywek. Koordynator ma jasny plan działań operacyjnych. Drużyny i trenerzy z wyprzedzeniem znają harmonogram.

UC-04: Ustalenie sponsorów i reklamy

Aktorzy: PZPS (jedyne akty); relacja «extends» względem Ustalenia terminu rozgrywki

Ciąg podstawowy: W trakcie lub po ustaleniu terminu rozgrywki, gdy spełnione są określone warunki (np. mecz rangi krajowej, finał), PZPS opcjonalnie uruchamia podproces pozyskiwania sponsorów. Pracownik PZPS wprowadza do systemu dane sponsorów przypisanych do danego wydarzenia: nazwę sponsora, typ reklamy (banner, spot, nazwa na koszulkach), budżet.

Ciąg alternatywny / awaryjny: Jeśli sponsor wycofa się przed meczem, system umożliwia usunięcie wpisu i powiadomienie odpowiednich służb. Przypadek nie jest wywoływany dla meczów niższych lig bez umów sponsorskich.

Częstotliwość: Kilka-kilkanaście razy w sezonie (głównie dla meczów ekstraklasy i pucharowych). Czas typowy: 20 minut. Czas maksymalny: 1 godzina.

Wartość dla aktorów: PZPS systematyzuje informacje o sponsoringu i zapewnia właściwą ekspozycję reklamową podczas meczów, co przekłada się na wywiązanie się z umów sponsorskich.

UC-05: Zorganizuj mecz

Aktorzy: Koordynator (główny wykonawca)

Ciąg podstawowy: Po ustaleniu terminu system automatycznie inicjuje ten przypadek użycia. Koordynator w systemie przechodzi przez listę kontrolną organizacji meczu: potwierdza obsadę sędziowską, sprawdza dostępność sprzętu, weryfikuje przyjazd drużyn. Oznacza kolejne punkty listy jako zrealizowane. System rejestruje status gotowości meczu.

Ciąg alternatywny / awaryjny: Jeśli jeden z elementów listy kontrolnej nie może zostać spełniony (np. choroba sędziego), system eskaluje problem do PZPS z propozycją rozwiązania. Jeśli mecz nie osiągnie statusu „gotowy do rozegrania” na 24h przed terminem, system generuje alert.

Częstotliwość: Raz na każdy zaplanowany mecz. Czas typowy: 30–60 minut (rozłożone na kilka dni). Czas maksymalny: kilka godzin w dniu meczu przy problemach organizacyjnych.

Wartość dla aktorów: Koordynator ma pełną kontrolę nad gotowością operacyjną meczu. PZPS zyskuje pewność, że spotkania są właściwie przygotowane.

UC-06: Rezerwację hali sportowej

Aktorzy: Koordynator; wywoływany przez Ustalenie terminu rozgrywki («includes»)

Ciąg podstawowy: System automatycznie inicjuje rezerwację po zatwierdzeniu terminu meczu. Koordynator w systemie wybiera halę sportową z bazy obiektów, sprawdza jej dostępność w kalendarzu i dokonuje rezerwacji na wymagany czas (zazwyczaj termin meczu + 2h na przygotowanie). System wysyła automatyczne potwierdzenie do zarządcy hali i zapisuje rezerwację.

Ciąg alternatywny / awaryjny: Jeśli hala jest już zajęta, system wyświetla listę dostępnych obiektów w tym samym mieście lub regionie. Jeśli rezerwacja zostanie anulowana przez zarządcę hali, system powiadamia Koordynatora i PZPS o konieczności znalezienia zastępczego obiektu.

Częstotliwość: Raz na każdy mecz (tyle samo co UC-06). Spiętrzenia przy planowaniu całego sezonu (jednorazowo kilkaset rezerwacji). Czas typowy: 5 minut. Czas maksymalny: 30 minut przy trudnościach z dostępnością.

Wartość dla aktorów: Koordynator ma pewność co do miejsca rozegrania meczu. PZPS zyskuje udokumentowane rezerwacje obiektów, minimalizując ryzyko kolizji.

UC-07: Przydział drużyny i trenerów

Aktorzy: Koordynator (inicjator), Trener (uczestnik); powiązane «includes»: Wystawienie Drużyny, Wyświetl fragment nagrania z meczu, Podejrzyj statystyki zawodnika, Zobacz podgląd taktyczny

Ciąg podstawowy: PZPS inicjuje przydziały na początku sezonu lub w trakcie jego trwania (np. play-offy). Pracownik PZPS w systemie przegląda listę zarejestrowanych drużyn i trenerów z aktywnymi licencjami. Przypisuje drużyny do lig i grup rozgrywkowych, a trenerów do drużyn. System weryfikuje poprawność przydziałów (aktywna licencja, brak konfliktów). Po zatwierdzeniu generowane są powiadomienia dla zainteresowanych trenerów.

Ciąg alternatywny / awaryjny: Jeśli trener nie posiada ważnej licencji, system blokuje przydział i wyświetla ostrzeżenie. Jeśli drużyna nie spełnia wymagań ligowych (np. za mała kadra), system flaguje problem przed zatwierdzeniem.

Częstotliwość: Głównie raz na sezon (sierpień–wrzesień), z drobnymi korektami w trakcie roku. Czas typowy: 1–2 godziny na przydziały dla całej ligi. Czas maksymalny: pół dnia roboczego.

Wartość dla aktorów: PZPS zapewnia kompletność i zgodność obsady ligowej z regulaminem. Trenerzy uzyskują formalne potwierdzenie prowadzenia drużyny i dostęp do narzędzi systemowych.

UC-08: Wystawienie Drużyny

Aktorzy: Trener; wywoływany przez Przydział drużyny i trenerów («includes»)

Ciąg podstawowy: Przed meczem Trener loguje się do systemu i otwiera formularz składu meczowego. Z listy zawodników swojej drużyny wybiera maksymalnie 14 graczy (zgodnie z przepisami siatkówki), wyznacza libero, kapitana i ewentualnych zmienników. Zatwierdza skład. System sprawdza poprawność formalną (liczba zawodników, ważność licencji graczy) i zapisuje skład meczowy, udostępniając go sędziom i PZPS.

Ciąg alternatywny / awaryjny: Jeśli zawodnik jest kontuzjowany (status w systemie) lub zdyskwalifikowany, system ostrzega trenera przed zatwierdzeniem składu. Jeśli skład zostanie złożony po wymaganym terminie (np. 1h przed meczem), system rejestruje opóźnienie i powiadamia koordynatora.

Częstotliwość: Przed każdym meczem drużyny, czyli kilka razy tygodniowo w sezonie. Czas typowy: 5–10 minut. Czas maksymalny: 20 minut.

Wartość dla aktorów: Trener formalizuje decyzję taktyczną dotyczącą składu. System i sędziowie uzyskują oficjalny, zatwierdzony skład meczowy.

UC-09: Zobacz podgląd taktyczny

Aktorzy: Trener; wywoływany przez Przydział drużyny i trenerów («includes»)

Ciąg podstawowy: Trener po zalogowaniu wybiera opcję analizy taktycznej. System wyświetla narzędzie do rysowania ustawień na boisku (schematyczne boisko siatkówki z pozycjami). Trener tworzy lub przegląda zapisane schematy taktyczne (rotacje, przyjęcia, ataki). Może eksportować schemat do PDF lub udostępnić zawodnikom. System zapisuje tworzone schematy w profilu drużyny.

Ciąg alternatywny / awaryjny: Jeśli trener próbuje edytować schemat drużyny, do której nie jest przypisany, system odmawia dostępu. Jeśli narzędzie rysowania nie ładuje się poprawnie (błąd JS), system proponuje wersję uproszczoną (statyczne diagramy).

Częstotliwość: Kilka razy tygodniowo, intensywniej w tygodniu poprzedzającym mecz. Czas typowy: 15–30 minut na sesję taktyczną. Czas maksymalny: brak ograniczenia czasowego.

Wartość dla aktorów: Trener zyskuje cyfrowe narzędzie do przygotowania taktycznego, zastępujące tradycyjne tablice. Zawodnicy mogą przeglądać schematy przed meczem.

UC-10: Przypisanie numerów startowych

Aktorzy: Trener

Ciąg podstawowy: Trener po wystawieniu składu meczowego otwiera moduł numeracji. System wyświetla listę wybranych zawodników z polami na numery koszulek. Trener przypisuje każdemu zawodnikowi numer (1–99, zgodnie z przepisami FIVB). System waliduje unikalność numerów w obrębie drużyny i zapisuje przypisania. Dane trafiają do protokołu meczowego.

Ciąg alternatywny / awaryjny: Jeśli zawodnik ma numer przypisany na stałe w swoim profilu, system automatycznie go podpowiada. Jeśli dwa mecze toczą się jednocześnie, system pozwala na różne numery koszulek dla tego samego zawodnika w różnych protokołach.

Częstotliwość: Przed każdym meczem, często łącznie z wystawianiem składu. Czas typowy: 3–5 minut. Czas maksymalny: 10 minut.

Wartość dla aktorów: Trener formalizuje numery startowe wymagane przez przepisy. Sędziowie i sekretarz meczu uzyskują kompletny protokół startowy.

UC-11: Przydział sędziów

Aktorzy: Koordynator

Ciąg podstawowy: Koordynator otwiera listę zaplanowanych meczów bez kompletnej obsady sędziowskiej. Dla każdego meczu wybiera z listy dostępnych i licencjonowanych sędziów (I sędzia, II sędzia, sędziowie liniowi, sekretarz). System sprawdza dostępność sędziego (brak konfliktu terminowego, aktywna licencja). Po zatwierdzeniu system wysyła powiadomienie do przydzielonych sędziów z danymi meczu.

Ciąg alternatywny / awaryjny: Jeśli żaden licencjonowany sędzia nie jest dostępny w danym terminie i regionie, system rozszerza wyszukiwanie na sąsiednie regiony lub generuje alert niedoboru kadrowego. Jeśli sędzia odmówi przydziału, Koordynator musi wybrać zastępstwo.

Częstotliwość: Kilka–kilkanaście razy tygodniowo w sezonie; masowe spiętrzenia przy ustalaniu obsad na początku sezonu. Czas typowy: 5–10 minut na jeden mecz. Czas maksymalny: 30 minut przy trudnej dostępności.

Wartość dla aktorów: Koordynator zapewnia kompletną i legalną obsadę sędziowską meczów. PZPS zyskuje pewność zgodności z regulaminem. Sędziowie uzyskują z wyprzedzeniem informację o przydzielonych im spotkaniach.

UC-12: Wypożycz sprzęt

Aktorzy: Koordynator

Ciąg podstawowy: Koordynator otwiera moduł zarządzania sprzętem. Dla planowanego meczu wybiera potrzebny ekwipunek z magazynu PZPS (siatka meczowa, słupki, piłki, tablice wynikowe, sprzęt nagłaśniający). System sprawdza dostępność sprzętu w danym terminie i lokalizacji. Koordynator potwierdza rezerwację. System rejestruje wydanie sprzętu i wymagany termin zwrotu.

Ciąg alternatywny / awaryjny: Jeśli wymagany sprzęt jest niedostępny (wypożyczony na inny mecz), system sugeruje wypożyczenie zewnętrzne lub transport z innej lokalizacji. Po terminie zwrotu system automatycznie generuje przypomnienie.

Częstotliwość: Raz na każdy mecz wymagający sprzętu z zasobów PZPS (nie wszystkie hale mają własny kompletny sprzęt). Czas typowy: 10 minut. Czas maksymalny: 30 minut.

Wartość dla aktorów: Koordynator ma kontrolę nad logistyką sprzętową. PZPS ogranicza ryzyko zagubienia lub uszkodzenia kosztownego wyposażenia.

UC-13: Odwołaj /przesuń mecz

Aktorzy: Koordynator

Ciąg podstawowy: Na skutek decyzji PZPS lub siły wyższej (warunki pogodowe, awaria hali, pandemia) Koordynator otwiera zaplanowany mecz w systemie i wybiera opcję „Odwołaj” lub „Przesuń”. W przypadku przesunięcia wprowadza nowy termin i lokalizację. System automatycznie anuluje powiązane rezerwacje (hala, sędziowie, sprzęt), generuje powiadomienia do drużyn, trenerów i sędziów oraz aktualizuje terminarz.

Ciąg alternatywny / awaryjny: Jeśli nowy termin koliduje z innymi zaplanowanymi meczami na tej hali, system informuje Koordynatora i wymaga wyboru alternatywnego obiektu. Jeśli mecz jest odwoływany mniej niż 48h przed terminem, system generuje alert o możliwych karach finansowych zgodnie z regulaminem.

Częstotliwość: Rzadko w normalnych warunkach, masowo w sytuacjach kryzysowych. Czas typowy: 15 minut. Czas maksymalny: 1 godzina

Wartość dla aktorów: Koordynator dokumentuje decyzję organizacyjną w systemie. Wszystkie zainteresowane strony (drużyny, sędziowie, hale) są natychmiastowo i automatycznie powiadamiane.

UC-14: Zarządzaj dostępnymi halami

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję zarządzania halami. Wyświetlana jest lista wszystkich zarejestrowanych obecnie hal, wraz z możliwością wyszukiwania oraz przycisk dodania nowej hali. Użytkownik wybiera interesującą go opcję dodania nowej hali, zaktualizowania informacji o jednej hali z listy bądź usunięcia wybranej hali. Użytkownik wypełnia odpowiedni formularz z danymi na temat nowej/wybranej hali. Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Użytkownik w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Kilka razy do roku. Dosyć rzadkie spiętrzenia, tylko przy znaczących zmianach w infrastrukturze. Czas typowy: od kilku do kilkunastu minut. Czas maksymalny: 30 minut

Wartość dla aktorów: Wyświetlany jest komunikat o prawidłowym wykonaniu operacji, do systemu zostaje dodana nowa hala lub informacje o niej zostają zaktualizowane.

UC-15: Zarządzaj dostępnym sprzętem

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję zarządzania sprzętem. Użytkownik wybiera halę, w której sprzęt jest/będzie dostępny. Wyświetlana jest lista sprzętu dostępnego w danej hali, wraz z możliwością wyszukiwania oraz przycisk dodania nowego sprzętu. Użytkownik wybiera interesującą go opcję dodania nowego sprzętu, zaktualizowania informacji o jednym ze sprzętów z listy bądź usunięcia go. Użytkownik wypełnia odpowiedni formularz z danymi na temat nowego/wybranego sprzętu. Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Użytkownik w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Kilka razy w miesiącu. Spiętrzenia przy masowych inwestycjach w sprzęt bądź wymianie go na nowszy. Czas typowy: od kilku do kilkadziesiąt minut. Czas maksymalny: do kilku godzin

Wartość dla aktorów: Wyświetlany jest komunikat o prawidłowym wykonaniu operacji, do systemu zostaje dodany nowy sprzęt lub informacje o nim zostają zaktualizowane.

UC-16: Dodaj użytkownika

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję dodania nowego użytkownika. Wypełnia formularz z danymi nowego użytkownika. Administrator ustawia uprawnienia nowego użytkownika (przypadek użycia „Ustaw uprawnienia użytkownika”) i zostaje wykonana dodatkowa weryfikacja tożsamości administratora. Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Administrator w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Od kilku do kilkunastu razy w miesiącu. Spiętrzenia przy rejestracji nowych zespołów, przy zatrudnianiu dużej ilości nowej kadry. Czas typowy: kilkanaście minut. Czas maksymalny: kilkadziesiąt minut

Wartość dla aktorów: Do systemu zostaje dodany nowy użytkownik.

UC-17: Ustaw uprawnienia użytkownika

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera w menu głównym panelu administracyjnego opcję ustawienia uprawnień użytkownika. Wyświetlona zostaje lista użytkowników z możliwością wyszukiwania, skąd wybierany jest modyfikowany użytkownik, bądź administrator zostaje tu przekierowany z przypadku użycia „Dodaj użytkownika”. Administrator ustawia nową rolę użytkownika (administrator, koordynator, statystyk, trener, zawodnik). Zostaje wykonana dodatkowa weryfikacja tożsamości administratora (przypadek użycia „Weryfikuj tożsamość administratora”). Dane zostają zaktualizowane w bazie danych.

Ciąg alternatywny / awaryjny: Administrator w pewnym momencie anulował wprowadzanie danych – następuje powrót do poprzedniego menu. Weryfikacja uprawnień administratora nie powiodła się – wyświetlony zostaje komunikat błędu.

Częstotliwość: Od kilku do kilkunastu razy w miesiącu. Spiętrzenia przy rejestracji nowych zespołów, przy zatrudnianiu dużej ilości nowej kadry. Czas typowy: kilka minut. Czas maksymalny: 10 minut

Wartość dla aktorów: Rola użytkownika zostaje zmieniona, wyświetlana jest informacja o prawidłowym przeprowadzeniu operacji.

UC-18: Weryfikuj tożsamość administratora

Aktorzy: Administrator

Ciąg podstawowy: Wyświetlony zostaje ekran weryfikacji tożsamości administratora. Administrator wprowadza swoje specjalne ID/hasło, skanuje swoją kartę.

Ciąg alternatywny / awaryjny: Zostają wprowadzone nieprawidłowe dane – zostaje zgłoszony błąd.

Częstotliwość: Do kilku razy w tygodniu. Spiętrzenia przy masowych inwestycjach, zmianach kadry. Czas typowy: do jednej minuty. Czas maksymalny: do dziesięciu minut

Wartość dla aktorów: Tożsamość administratora zostaje zweryfikowana i kontynuowane jest wykonywanie akcji wymagającej danych uprawnień.

UC-19: Przygotuj analizę transmisji wideo

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera przycisk rozpoczęcia nowego meczu. Do systemu zostają kolejno podpięte i skonfigurowane kamery bądź pliki wideo. Następuje kalibracja i weryfikacja, czy system działa poprawnie. Wypełniany jest protokół z informacjami dotyczącymi meczu, który ma się odbyć. Tuż przed rozpoczęciem meczu administrator naciska przycisk rozpoczynający analizę.

Ciąg alternatywny / awaryjny: Następuje błąd podczas konfiguracji kamer – po rozwiązaniu problemu, administrator ponawia weryfikację działania systemu.

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilka godzin. Czas maksymalny: do pięciu godzin

Wartość dla aktorów: Do systemu zostają podłączone kamery, rozpoczyna się analiza obrazu z kamer przez AI, otwierany jest główny interfejs analizy meczu.

UC-20: Przygotuj integrację z systemem transmisyjnym TV

Aktorzy: Administrator

Ciąg podstawowy: Administrator wybiera opcję przekierowania wideo z kamer do analizy w celu transmisji TV. Wybierana i konfigurowana jest wybrana nakładka prezentująca wyniki meczu. Pracownik telewizji bądź serwisu online podpina i konfiguruje sprzęt do transmisji.

Ciąg alternatywny / awaryjny: Następuje błąd podczas konfiguracji systemu transmisyjnego – administrator i pracownik telewizji bądź serwisu online rozwiązują problem i kontynuują konfigurację.

Częstotliwość: Co kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: do pięciu godzin

Wartość dla aktorów: Rozpoczyna się przekazywanie transmisji na żywo z nałożoną odpowiednią nakładką do stacji telewizyjnej lub serwisu online.

UC-21: Zapisz dane po meczu do ligowej bazy danych

Aktorzy: Administrator, PZPS (Polski Związek Piłki Siatkowej)

Ciąg podstawowy: Po zakończeniu meczu wyświetlane jest jego podsumowanie. Dokonywana jest dodatkowa weryfikacja zebranych statystyk. Administrator zatwierdza zebrane dane, które zostają następnie przesłane do ligowej bazy danych.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: do kilku godzin

Wartość dla aktorów: Do ligowej bazy danych zostają wpisane informacje na temat meczu zebrane w trakcie działania systemu.

UC-22: Obserwuj analizę dokonywaną przez AI na żywo

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk potwierdza swoją tożsamość na urządzeniu, na którym będzie weryfikował dane. Po rozpoczęciu meczu wyświetlany jest panel główny analizy meczu wraz z m.in. heatmapą, listą zarejestrowanych akcji i podstawowymi statystykami. Statystyk obserwuje i weryfikuje poprawność pracy systemu.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: 90-120 minut. Czas maksymalny: kilka godzin

Wartość dla aktorów: Statystyk widzi wizualizację pracy systemu i zbierane przez system w czasie rzeczywistym dane.

UC-23: Generuj raport z meczu

Aktorzy: Statystyk

Ciąg podstawowy: Po zakończeniu meczu zostaje wyświetlone menu podsumowania meczu. Statystyk jeszcze raz weryfikuje integralność zebranych danych i statystyk. Statystyk wybiera przycisk wygenerowania raportu. Zostaje wyświetlony formularz do raportu, statystyk uzupełnia brakujące dane i podaje opis meczu. Po naciśnięciu przycisku zatwierdzającego zostaje wygenerowany gotowy do pobrania raport w formacie PDF.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Co 1-kilka dni. Spiętrzenia w środku sezonu. Czas typowy: kilkadziesiąt minut. Czas maksymalny: kilka godzin

Wartość dla aktorów: Raport z meczu z najważniejszymi statystykami i ogólnym opisem w formacie PDF.

UC-24: Podejrzuj statystykę/akcję w trakcie trwania meczu

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk podczas analizy meczu naciska przycisk wyświetlający szczegóły na temat danej statystyki/akcji. Ukazuje się okno pokazujące szczegóły na temat danej statystyki/akcji, jej ostatnich zmian wraz z ich powodami. Dostępne są również przyciski służące do powrotu oraz do korekcji statystyki.

Ciąg alternatywny / awaryjny: –

Częstotliwość: Kilkadziesiąt razy w trakcie jednego meczu. Spiętrzenia w trakcie meczu. Czas typowy: ok. minuty. Czas maksymalny: do pięciu minut

Wartość dla aktorów: Wyświetlone zostają szczegółowe dane pokazujące zmiany danej statystyki w czasie i akcje, które wpłynęły na jej zmianę.

UC-25: Skoryguj wybraną statystykę/akcję

Aktorzy: Statystyk

Ciąg podstawowy: Statystyk podczas analizy meczu naciska przycisk wyświetlający szczegóły na temat danej statystyki/akcji. Ukazuje się okno pokazujące szczegóły na temat danej statystyki/akcji, jej ostatnich zmian wraz z ich powodami. Statystyk wybiera przycisk umożliwiający jej korektę. Statystyk wprowadza w wyznaczonym miejscu prawidłową wartość statystyki bądź zmienia typ rozpoznanej akcji, po czym zapisuje zmiany. Następuje powrót do panelu głównego analizy.

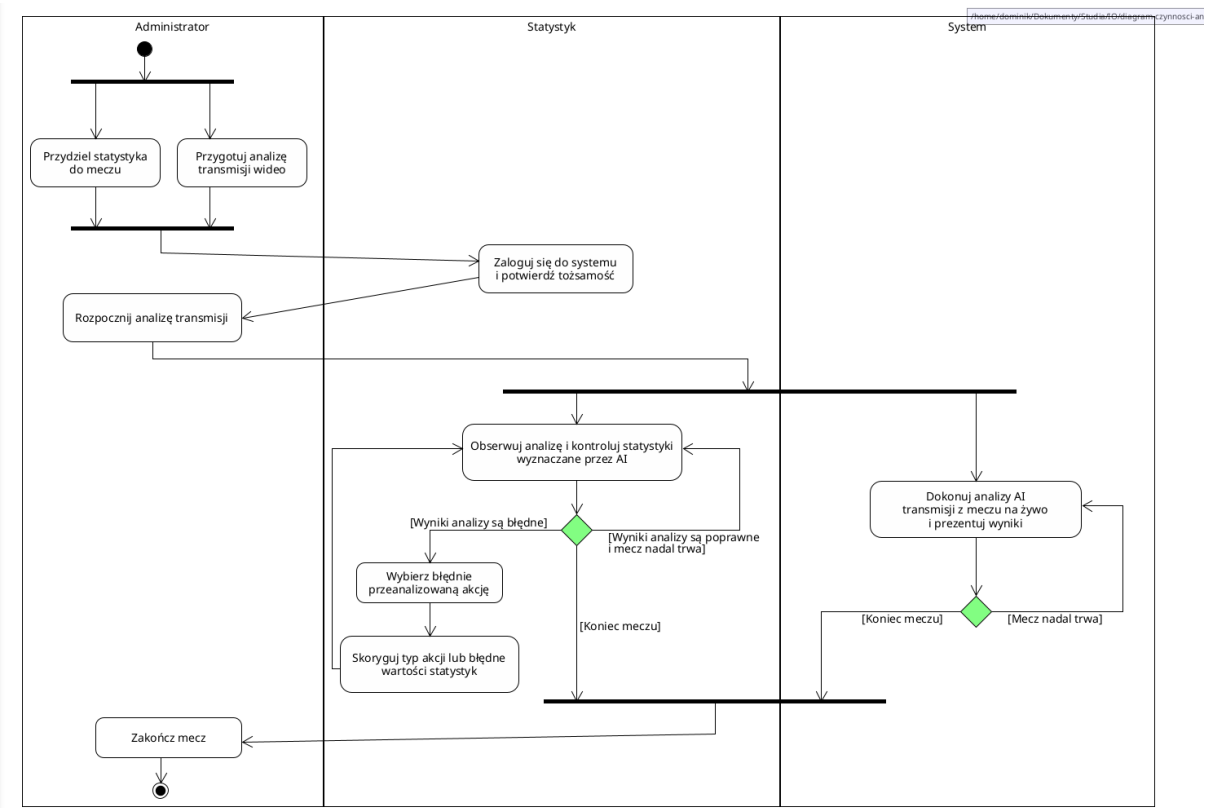
Ciąg alternatywny / awaryjny: –

Częstotliwość: Do kilkunastu razy w trakcie jednego meczu. Spiętrzenia w trakcie meczu. Czas typowy: ok. minuty. Czas maksymalny: do pięciu minut

Wartość dla aktorów: Wartość wybranej statystyki lub rodzaj akcji zostają skorygowane po błędnym automatycznym rozpoznaniu.

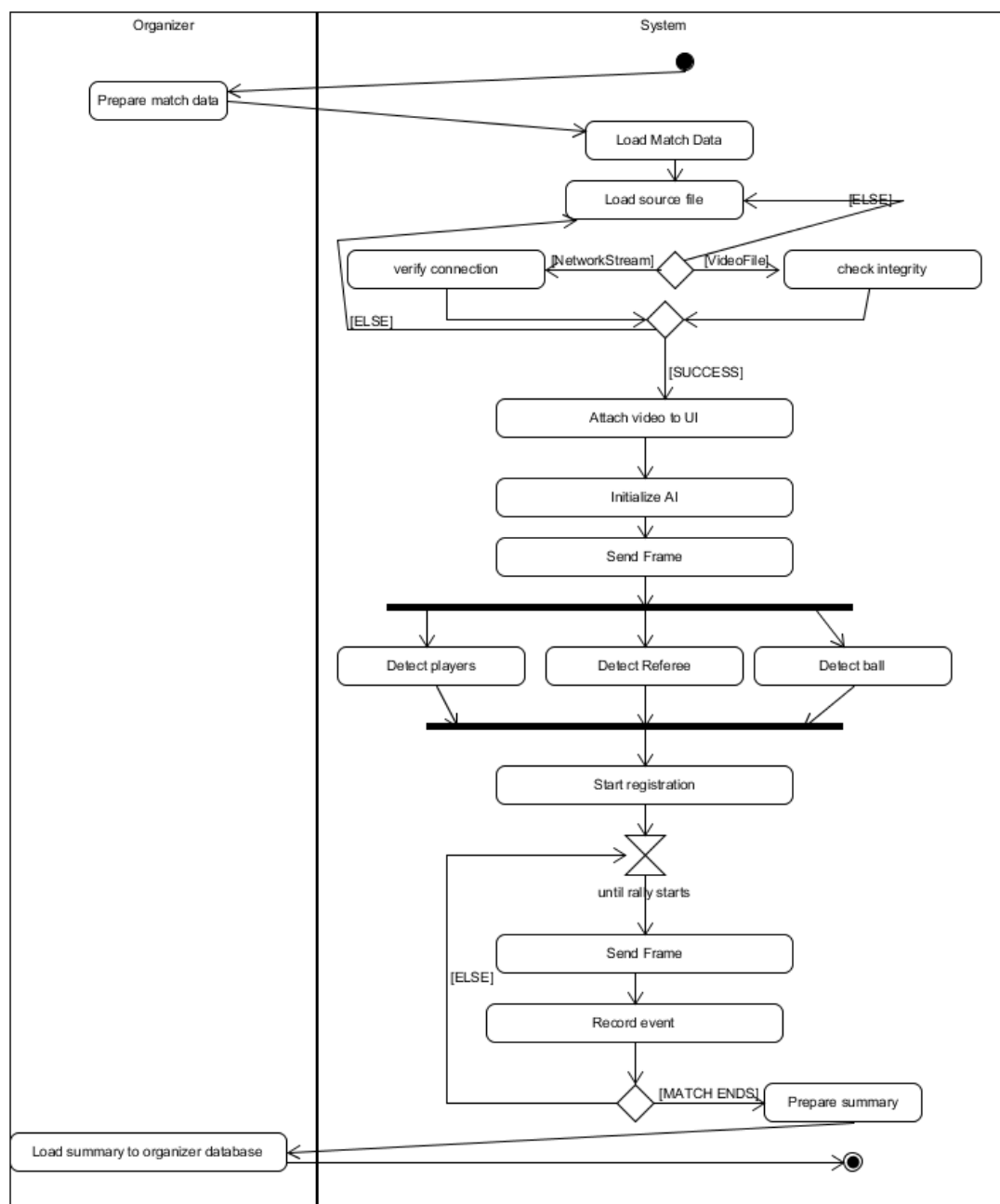
Diagramy czynności

Dominik Jarzyło



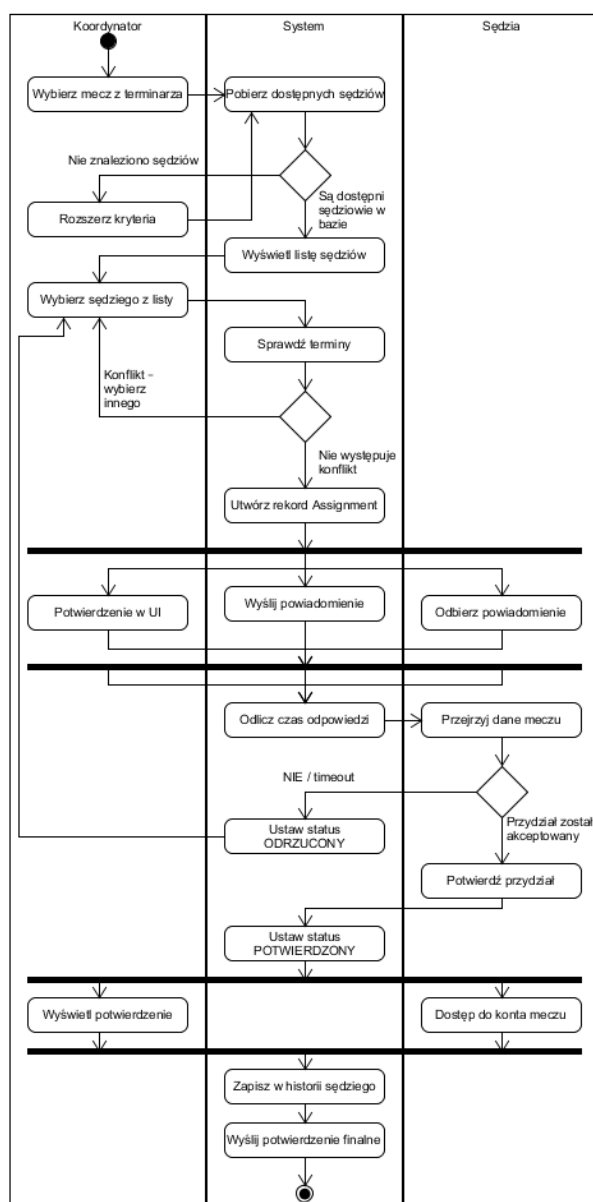
Rysunek 3 Diagram czynności dla przypadku użycia „Obserwuj analizę dokonywaną przez AI na żywo”

Karol Kuc



Rysunek 4 Diagram czynności dla przypadku użycia "Przygotuj analizę transmisji wideo"

Mateusz Tręda



Rysunek 5 Diagram czynności dla przypadku użycia „Przydział sędziów”

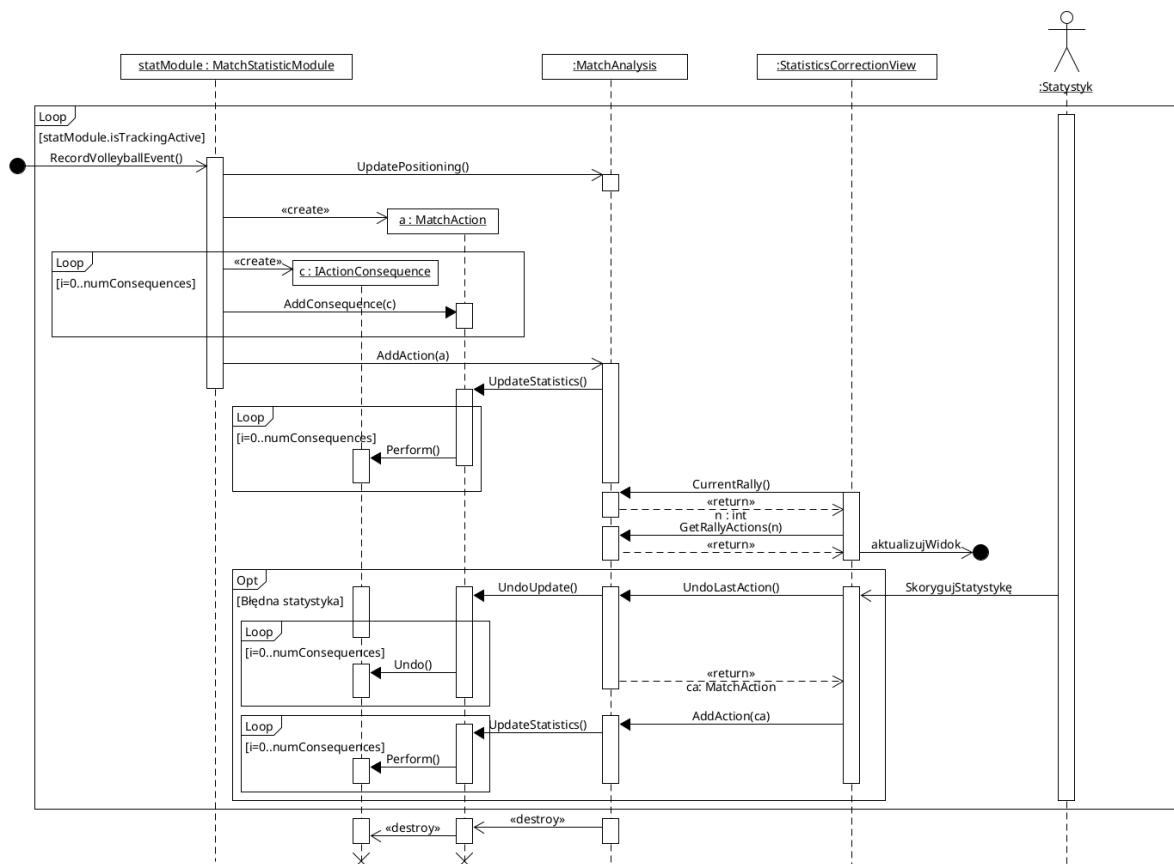
Diagram jest podzielony na trzy partycje, Koordynator, System i Sędzia. Ścieżka „System” zawiera sekwencję działań i punktów decyzyjnych. Wpierw Koordynator wybiera mecz, do którego ma być przydzielony sędzia. Później Koordynator wykonuje request pobrania listy sędziów na podstawie swoich kryteriów. Jeśli są dostępni to może ich wybrać z listy sprawdzając konflikty z terminami.

Po wybraniu sędziego tworzy się obiekt Assignment czyli Przydział sędziego. Do sędziego jest wysyłane powiadomienie o przydziale. Operacja jest widoczna dla wszystkich 3 partycji. Sędzia ma dostęp do danych meczu, gdzie jest przydzielany i ma możliwość akceptować lub odrzucić. Status Przydziału jest potwierdzony a sędzia otrzymuje dostęp do konta analizy meczu.

Na końcu wszystko jest zapisane w historii i jest wykonane finalne potwierdzenie w Emailu sędziego.

Diagramy przebiegu

Dominik Jarzyło

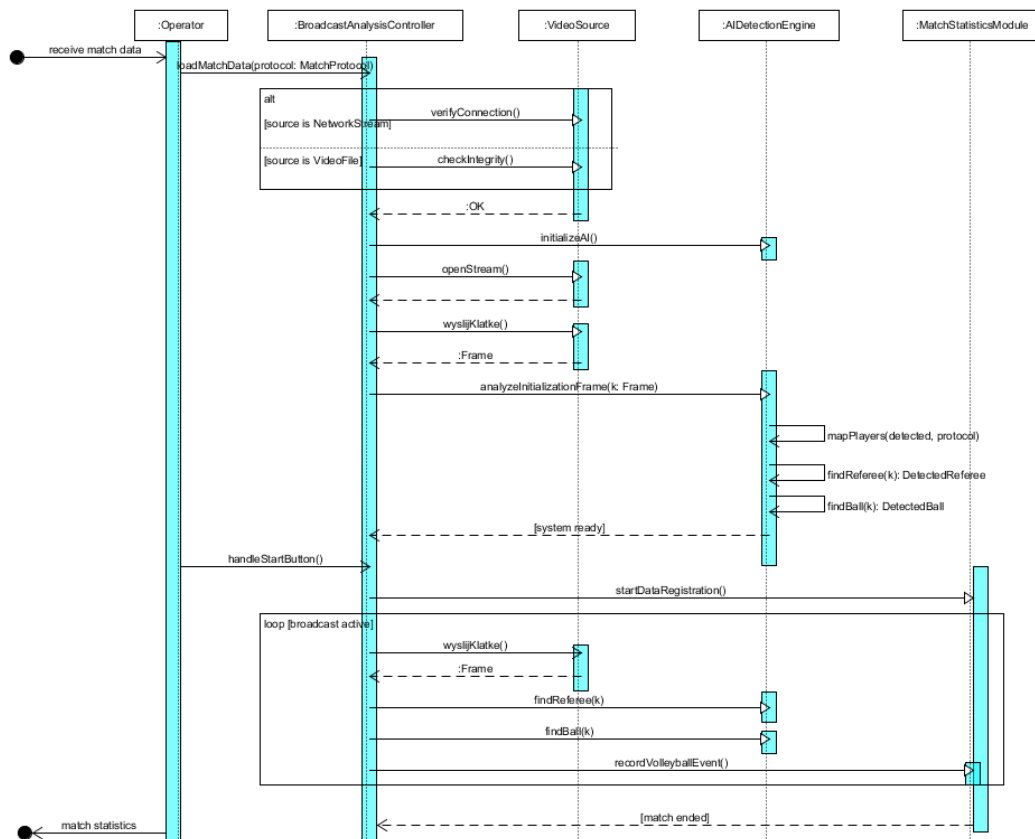


Rysunek 6 Diagram przebiegu dla przypadku użycia „Obserwuj analizę dokonywaną przez AI na żywo”

Komunikaty odbierane i wysyłane na zewnątrz fragmentu systemu:

- RecordVolleyballEvent() – wysyłany do obiektu klasy MatchStatisticsModule, gdy system analizy AI zidentyfikuje pewne wydarzenie w celu dalszej analizy jego rodzaju oraz wpływu na statystyki,
- AktualizujWidok – komunikat powodujący zaktualizowanie danych wyświetlanych przez interfejs użytkownika,
- SkorygujStatystykę – komunikat wysyłany przez interfejs użytkownika do obiektu klasy StatisticsCorrectionView, gdy statystyk dokona korekcji pewnej akcji i związanych z nią statystyk. Powoduje to cofnięcie w systemie ostatnio rozpoznanej akcji, zmianę odpowiednich parametrów w obiekcie akcji i zarejestrowanie jej ponownie.

Karol Kuc



Rysunek 7 Diagram przebiegu dla przypadku użycia „Przygotuj analizę transmisji wideo”

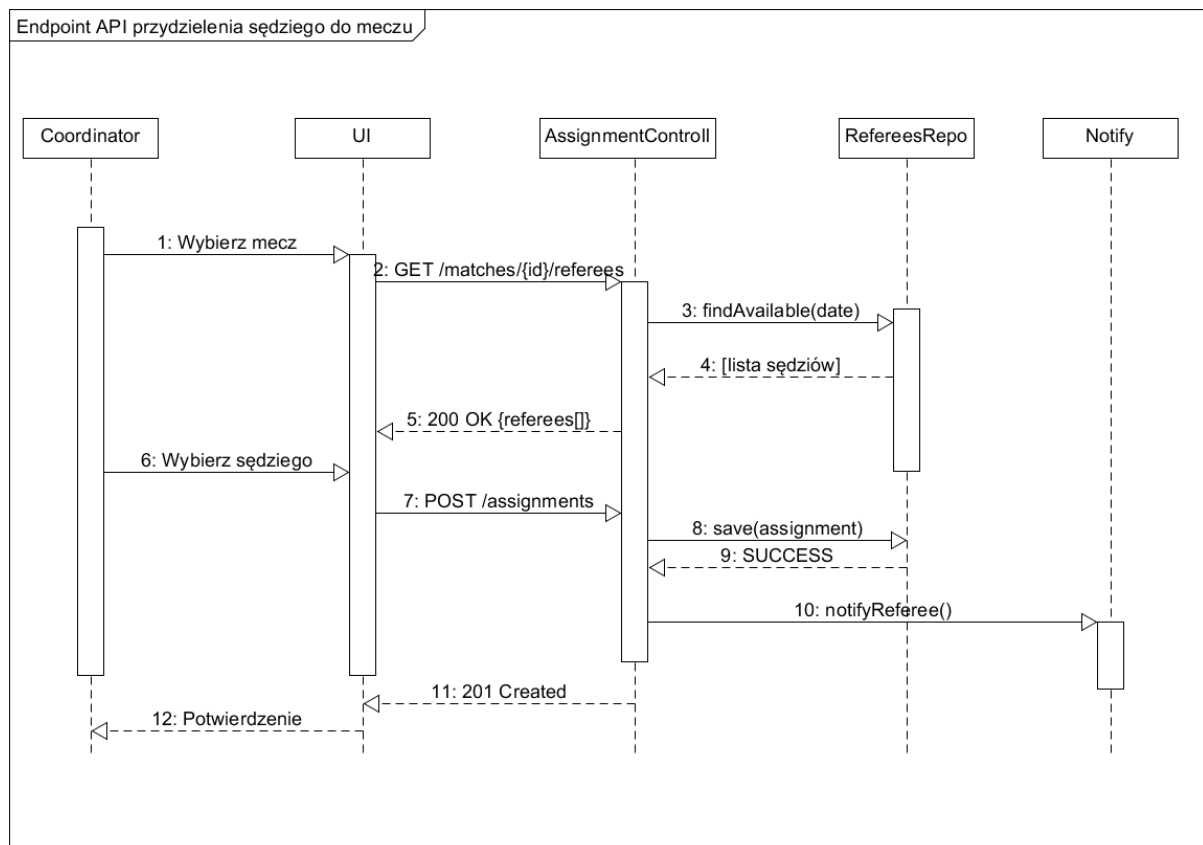
Poniższy diagram przebiegu przedstawia sekwencję komunikatów wymienianych między obiektami systemu podczas realizacji zdarzenia systemowego Przygotuj analizę transmisji wideo. Scenariusz obejmuje fazę inicjalizacji systemu, kalibrację silnika detekcji AI oraz rejestrację zdarzeń meczowych w trakcie transmisji.

Opis sekwencji komunikatów

1. **receive match data** → **Operator**: Zewnętrzny wyzwalacz (oznaczony stanem początkowym na diagramie) informuje operatora o dostępności danych meczowych do przetworzenia.
2. **Operator** → **:BroadcastAnalysisController**: `loadMatchData(protocol: MatchProtocol)` — Operator inicjuje scenariusz, przekazując do kontrolera obiekt protokołu meczowego zawierający identyfikator meczu, datę, nazwę areny, składy drużyn oraz listę sędziów.
3. **Fragment *alt*** [typ źródła wideo] — Kontroler weryfikuje typ podłączonego źródła wideo. Możliwe są dwa warianty:
 - [source is NetworkStream]: **:BroadcastAnalysisController** → **:VideoSource**: `verifyConnection()` — asynchroniczne wywołanie weryfikujące dostępność połączenia sieciowego ze strumieniem.
 - [source is VideoFile]: **:BroadcastAnalysisController** → **:VideoSource**: `checkIntegrity()` — asynchroniczne wywołanie sprawdzające spójność i dostępność pliku wideo.
4. **:VideoSource** → **:BroadcastAnalysisController**: `:OK` — Obiekt źródła wideo zwraca komunikat zwrotny potwierdzający pomyślną weryfikację. Kontroler może przejść do kolejnego etapu.
5. **:BroadcastAnalysisController** → **:AIDetectionEngine**: `initializeAI()` — Kontroler asynchronicznie inicjalizuje silnik detekcji AI, ładując modele detekcji
6. **:BroadcastAnalysisController** → **:VideoSource**: `openStream()` — Kontroler asynchronicznie poleca obiektowi źródła wideo otwarcie strumienia/pliku. Źródło wideo potwierdza gotowość komunikatem zwrotnym (linia przerywana).
7. **:BroadcastAnalysisController** → **:VideoSource**: `wyslijKlatke()` — Kontroler żąda przesłania klatki inicjalizacyjnej.
8. **:VideoSource** → **:BroadcastAnalysisController**: `:Frame` — Obiekt źródła wideo zwraca klatkę wideo (obiekt klasy `Frame`) zawierającą dane obrazu z rozdzielczością i znacznikiem czasu. Klatka ta posłuży do kalibracji silnika AI.
9. **:BroadcastAnalysisController** → **:AIDetectionEngine**: `analizeInitializationFrame(k: Frame)` — Kontroler asynchronicznie zleca silnikowi AI analizę klatki inicjalizacyjnej. Klatka jest przekazywana jako argument wywołania.
10. **Wywołania własne :AIDetectionEngine** — Silnik detekcji wykonuje trzy wewnętrzne operacje na klatce inicjalizacyjnej:
 - `mapPlayers(detected, protocol)` — mapuje wykrytych zawodników na rekordy z protokołu meczowego na podstawie numerów koszulek.
 - `findReferee(k): DetectedReferee` — lokalizuje sędziego na klatce i zwraca obiekt `DetectedReferee` z ramką detekcji.
 - `findBall(k): DetectedBall` — lokalizuje piłkę na klatce i zwraca obiekt `DetectedBall` z ramką detekcji.
11. **:AIDetectionEngine** → **:BroadcastAnalysisController**: `[system ready]` — Silnik AI zwraca komunikat przerywaną linią sygnalizujący ukończenie kalibracji. System jest gotowy do rejestracji meczu.

12. **Operator** → **:BroadcastAnalysisController:** handleStartButton() — Operator ręcznie naciska przycisk START, zlecając kontrolerowi uruchomienie rejestracji danych statystycznych.
13. **:BroadcastAnalysisController** → **:MatchStatisticsModule:** startDataRegistration() — Kontroler asynchronicznie uruchamia moduł statystyk. Od tego momentu system wchodzi w fazę właściwej analizy transmisji.

Mateusz Tręda



Rysunek 8 Diagram przebiegu dla endpointu api do przydzielenia sędziego do meczu

Uczestnicy (linie życia)

Koordynator — aktor ludzki inicjujący cały proces. Podejmuje decyzje: wybiera mecz i wskazuje konkretnego sędziego z otrzymanej listy. Jest pierwszym i ostatnim ogniwem przepływu.

UI — warstwa prezentacji (interfejs webowy/aplikacja). Pośredniczy między działaniami koordynatora a logiką backendową. Tłumaczy kliknięcia użytkownika na wywołania HTTP i prezentuje odpowiedzi systemu.

AssignmentControll — kontroler po stronie serwera (backend REST API). Główny orkiestrator logiki biznesowej: przyjmuje żądania HTTP, koordynuje pracę repozytorium i serwisu powiadomień, zwraca odpowiedzi do UI.

RefereesRepository (RefereeRepo) — warstwa dostępu do danych odpowiedzialna za sędziów. Odpytuje bazę danych w celu znalezienia dostępnych sędziów oraz utrwalenia nowego przydziału. Nie zawiera logiki biznesowej — wyłącznie operacje na danych.

NotificationService (Notify) — serwis odpowiedzialny za wysyłanie powiadomień. Wywoływany asynchronicznie po zapisaniu przydziału. Informuje sędziego o nowym zadaniu (e-mail / SMS).

Chronologiczny opis akcji

Akcja 1 — Koordynator → UI: „Wybierz mecz” Koordynator klika wybrany mecz w interfejsie. Jest to jedyne działanie czysto ludzkie inicjujące cały łańcuch. UI rejestruje zdarzenie i przechodzi do kroku 2.

Akcja 2 — UI → AssignmentController: GET /matches/{id}/referees UI wysyła żądanie HTTP GET do kontrolera, przekazując identyfikator wybranego meczu w ścieżce URL. Żądanie pyta: „kto jest dostępny do sędziowania tego meczu?”

Akcja 3 — AssignmentControll → RefereeRepo: findAvailable(date) Kontroler deleguje zapytanie do repozytorium, przekazując datę meczu jako parametr filtrowania. Repozytorium przeszukuje bazę danych pod kątem sędziów z aktywną licencją, bez konfliktu terminowego w tym dniu i z odpowiednim regionem.

Akcja 4 — JudgeRepo → AssignmentController: [lista sędziów] Repozytorium zwraca wynik zapytania — kolekcję obiektów sędziów spełniających kryteria. Komunikat oznaczony nawiasami kwadratowymi sygnalizuje, że jest to odpowiedź zwrotna (return), nie nowe wywołanie. Linia przerywana potwierdza charakter odpowiedzi.

Akcja 5 — AssignmentController → UI: 200 OK {judges[]} Kontroler odsyła do UI odpowiedź HTTP 200 z listą dostępnych sędziów w ciele odpowiedzi (JSON). UI renderuje listę — koordynator widzi kandydatów do wyboru.

Akcja 6 — Koordynator → UI: „Wybierz sędziego” Drugie i ostatnie działanie ludzkie w procesie. Koordynator wskazuje konkretną osobę z wyświetlonej listy i zatwierdza wybór. UI rejestruje decyzję i inicjuje zapis przydziału.

Akcja 7 — UI → AssignmentController: POST /assignments UI wysyła żądanie HTTP POST do kontrolera z danymi nowego przydziału w ciele żądania (identyfikator meczu + identyfikator sędziego). Użycie metody POST sygnalizuje tworzenie nowego zasobu.

Akcja 8 — AssignmentController → JudgeRepo: save(assignment) Kontroler przekazuje gotowy obiekt przydziału do repozytorium w celu utrwalenia go w bazie danych. Repozytorium wykonuje operację INSERT i zwraca potwierdzenie.

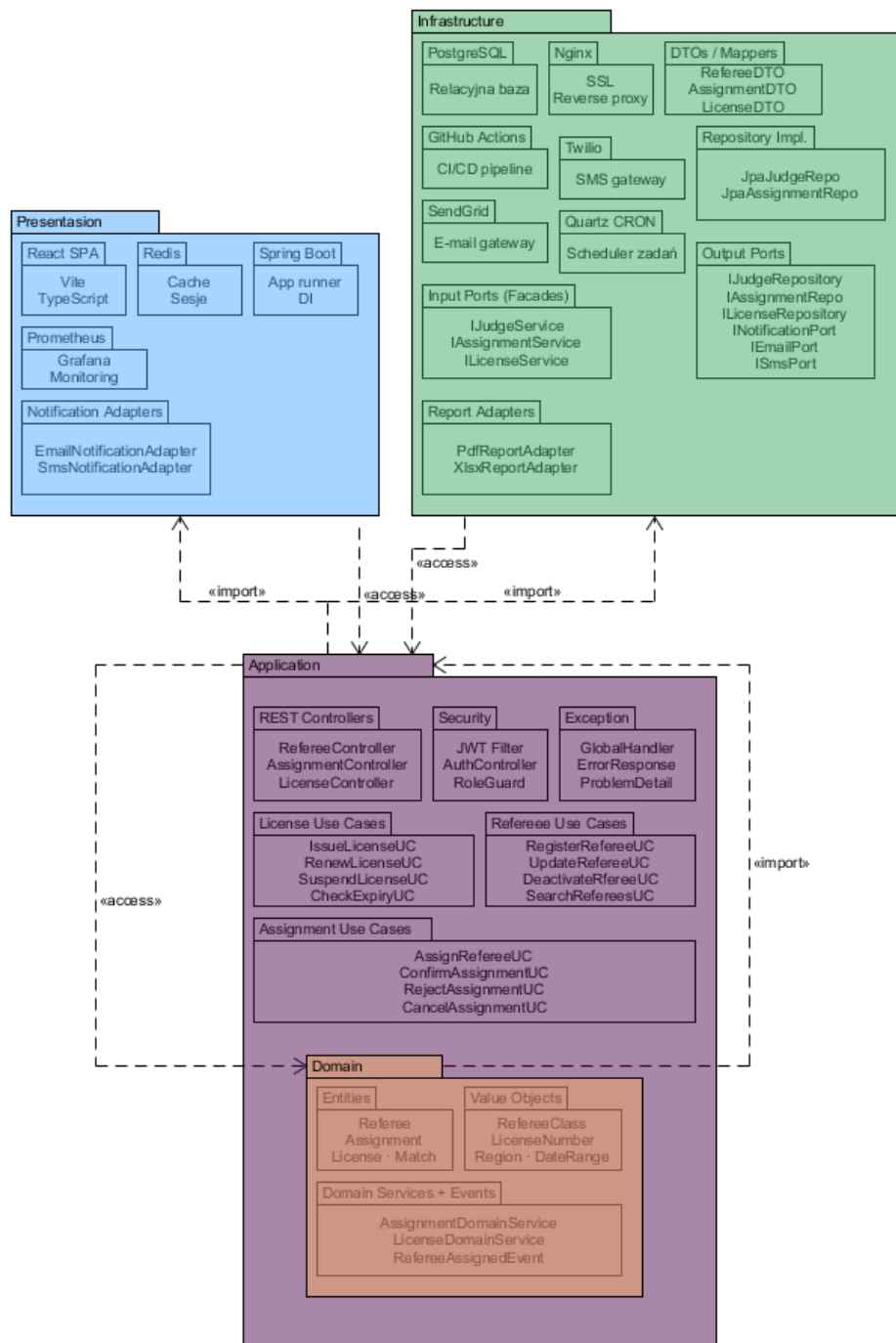
Akcja 9 — JudgeRepo → AssignmentController: SUCCESS Repozytorium potwierdza pomyślny zapis — przydział istnieje teraz w bazie danych ze statusem OCZEKUJĄCY. Linia przerywana oznacza komunikat zwrotny.

Akcja 10 — AssignmentController → NotificationService: notifyJudge() Po pomyślnym zapisie kontroler wywołuje serwis powiadomień. To wywołanie jest logicznie asynchroniczne — kontroler nie czeka na zakończenie wysyłki przed zwróceniem odpowiedzi do UI. Sędzia otrzymuje wiadomość e-mail lub SMS z danymi meczu.

Akcja 11 — AssignmentController → UI: 201 Created Kontroler zwraca do UI odpowiedź HTTP 201, sygnalizując pomyślne utworzenie nowego zasobu (przydziału). Odpowiedź może zawierać w nagłówku Location adres URL nowo powstałego rekordu.

Akcja 12 — UI → Koordynator: „Potwierdzenie” UI wyświetla koordynatorowi komunikat sukcesu — przydział został zapisany, sędzia powiadomiony. Proces dobiegł końca. Linia przerywana wskazuje, że jest to komunikat zwrotny kończący aktywację koordynatora.

5.) Perspektywa projektowa



Rysunek 9 Diagram pakietów

Opis architektury — Clean Architecture SZKS

Cała architektura opiera się na **Regule Zależności** (*Dependency Rule*): każda strzałka na diagramie wskazuje wyłącznie do wewnątrz. Domenowe encje nie wiedzą nic o bazie danych. Przypadki użycia nie wiedzą nic o HTTP. Framework jest wymienny.

Warstwa 1 — Enterprise Business Rules (domena)

Jądro systemu. Żadnych importów z zewnętrznych bibliotek — wyłącznie czyste klasy Java/Kotlin.

Entities — trwałe obiekty biznesowe z tożsamością i logiką wewnętrzną. Referee zawiera metodę `isAvailable(date)` i `isLicenseValid()`. Assignment zawiera metodę `confirm()` i `reject(reason)`. License zawiera `getDaysToExpiry()` i `isValid()`. Match zarządza statusem spotkania. Żadna z tych klas nie importuje JPA ani Springa.

Value Objects — niezmiennie obiekty bez tożsamości, równe przez wartość. RefereeClass (enum IV-MISTRZOWSKI) pilnuje reguł awansu. LicenseNumber waliduje format numeru licencji już w konstruktorze. Region standaryzuje nazewnictwo regionów. DateRange oblicza nakładanie się terminów.

Domain Services + Events — logika biznesowa, która nie należy do jednej encji. AssignmentDomainService sprawdza konflikty terminowe sędziego (logika wymaga zarówno Referee, jak i kalendarza). LicenseDomainService pilnuje reguł przedłużania. RefereeAssignedEvent to zdarzenie domenowe wyzwalające powiadomienia — domena wyrzuca zdarzenie, ale nie wie, kto je obsłuży.

Warstwa 2 — Application (Use Cases i Ports)

Orkiestracja przepływu. Wie o domenie, ale nie wie o bazie danych ani HTTP.

Use Cases — każdy przypadek użycia z diagramu UC jest osobną klasą. AssignRefereeUC wykonuje sekwencję: pobierz mecz → znajdź dostępnych sędziów → sprawdź konflikty (przez AssignmentDomainService) → zapisz przez port → wyrzuć event. Jedyna odpowiedzialność klasy to ten jeden przepływ.

Output Ports (interfejsy) — to czyste interfejsy Javy definiowane w tej warstwie, ale implementowane w warstwie 3. IRefereeRepository deklaruje `findAvailable(date): List<Referee>` i `save(Referee)`. INotificationPort deklaruje `notify(judge, match)`. Dzięki temu AssignRefereeUC nie zależy od JPA — zależy od interfejsu.

Input Ports (fasady) — interfejsy IRefereeService, IAssignmentService, ILicenseService eksponują publiczne API warstwy aplikacyjnej dla kontrolerów REST. Kontroler wstrzykuje interfejs, nie konkretną klasę.

Warstwa 3 — Interface Adapters

Tłumaczy między światem zewnętrznym a aplikacją. Wszystkie elementy tej warstwy są wymienne bez zmiany domeny.

REST Controllers — RefereeController, AssignmentController, LicenseController. Przyjmują żądania HTTP, mapują JSON → DTO, wywołują

odpowiedni Input Port (interfejs), zwracają odpowiedź HTTP. Nie zawierają logiki biznesowej.

DTOs i Mappers — obiekty transferu danych (np. RefereeDTO, AssignmentDTO) odizolowują kontrakt HTTP od encji domenowych. Mapper (np. MapStruct) konwertuje w obie strony. Dzięki temu zmiana struktury JSON nie wymaga zmiany domeny.

Security — JwtFilter weryfikuje token przed każdym żądaniem, RoleGuard sprawdza uprawnienia na poziomie metody (@PreAuthorize). Bezpieczeństwo jest adapterem — można je wymienić na OAuth2 bez dotykania logiki biznesowej.

Repository Implementations — JpaRefereeRepository implementuje interfejs IRefereeRepository z warstwy 2, używając Spring Data JPA i encji @Entity (osobnych od encji domenowych). To kluczowe: encja JPA to adapter, a nie encja domenowa.

Notification Adapters — EmailNotificationAdapter implementuje IEmailPort i wywołuje SendGrid SDK. SmsNotificationAdapter implementuje ISmsPort i wywołuje Twilio. Zamiana SendGrid na innego dostawcę = zmiana tylko tej klasy.

Report Adapters — PdfReportAdapter używa biblioteki iText, XlsxReportAdapter używa Apache POI. Oba implementują interfejsy zdefiniowane w warstwie 2.

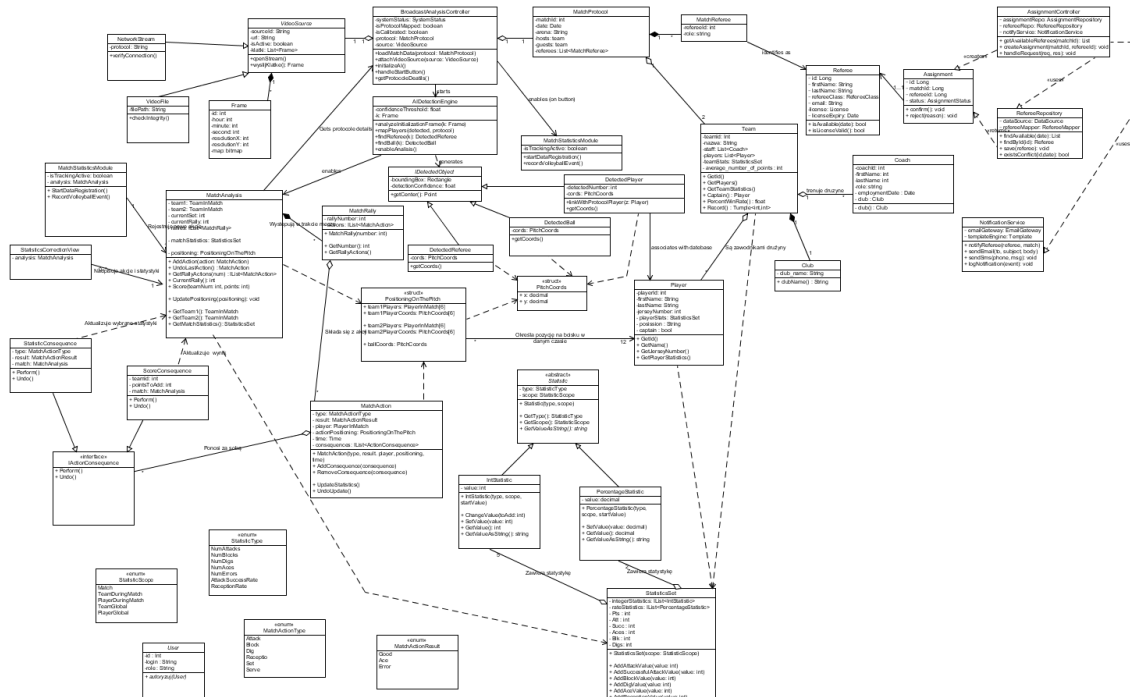
Exception Handler — GlobalExceptionHandler (@ControllerAdvice) mapuje wyjątki domenowe na kody HTTP i format ProblemDetail (RFC 9457). Logika błędów biznesowych nie wnika do kontrolerów.

Warstwa 4 — Frameworks & Drivers

Wszystko co zewnętrzne: bazy danych, serwery, narzędzia. Warstwa ta nic nie wie o domenie.

Na górze: **React SPA** budowany przez Vite, serwowany przez **Nginx** pełniący jednocześnie rolę reverse proxy i terminatora SSL. **Spring Boot** uruchamia kontener i wstrzykuje zależności — to on decyduje, która implementacja IRefereeRepository trafia do AssignRefereeUC. **PostgreSQL** to baza relacyjna. **Redis** przechowuje sesje JWT i cache wyników wyszukiwania.

Na dole: **SendGrid** i **Twilio** to zewnętrzne bramki komunikacyjne wywoływane przez adaptery. **Quartz CRON** codziennie o 8:00 odpala CheckExpiryUC. **GitHub Actions** buduje, testuje i deplojuje automatycznie przy każdym push. **Prometheus + Grafana** monitorują metryki JVM, bazy danych i API.



Rysunek 10 Diagram klas projektu

AIDetectionEngine

Klasa odpowiedzialna za analizę obrazu z wykorzystaniem sztucznej inteligencji. Wykrywa obiekty na klatkach wideo, takie jak zawodnicy, piłka oraz sędzia.

Atrybuty:

- confidenceThreshold: float – minimalny próg pewności, od którego detekcja obiektu jest uznawana za poprawną.
- k: Frame – aktualna klatka obrazu przekazywana do analizy.

Metody:

- `analyzeInitializationFrame(k: Frame)` – analizuje klatkę inicjalizacyjną w celu przygotowania systemu do pracy.
- `mapPlayers(detected, protocol)` – mapuje wykrytych zawodników na dane zawodników zapisane w protokole meczu.
- `findReferee(k): DetectedReferee` – wykrywa sędziego na podanej klatce obrazu.
- `findBall(k): DetectedBall` – wykrywa piłkę na podanej klatce obrazu.
- `enableAnalysis()` – uruchamia mechanizm analizy obrazu.

Assignment

Klasa reprezentująca przydział sędziego do konkretnego meczu.

Atrybuty:

- id: Long – unikalny identyfikator przydziału.
- matchId: Long – identyfikator meczu, którego dotyczy przydział.
- refereeId: Long – identyfikator przypisanego sędziego.
- status: AssignmentStatus – aktualny status przydziału.

Metody:

- confirm(): void – potwierdza przyjęcie przydziału przez sędziego.
- reject(reason): void – odrzuca przydział wraz z podaniem powodu.

AssignmentController

Klasa kontrolera odpowiedzialna za obsługę procesu przydzielania sędziego do meczu.

Atrybuty:

- assignmentRepo: AssignmentRepository – repozytorium przechowujące dane o przydziałach sędziów.
- refereeRepo: RefereeRepository – repozytorium danych sędziów.
- notifyService: NotificationService – serwis odpowiedzialny za wysyłanie powiadomień.

Metody:

- getAvailableReferees(matchId): List – zwraca listę sędziów dostępnych dla danego meczu.
- createAssignment(matchId, refereeId): void – tworzy nowy przydział sędziego do meczu.
- handleRequest(req, res): void – obsługuje żądanie użytkownika dotyczące przydziału sędziego.

BroadcastAnalysisController

Główna klasa sterująca procesem przygotowania i uruchomienia analizy transmisji wideo.

Atrybuty:

- `systemStatus`: `SystemStatus` – aktualny stan systemu.
- `isProtocolMapped`: `boolean` – informacja, czy dane z protokołu zostały poprawnie powiązane z analizą.
- `isCalibrated`: `boolean` – informacja, czy system został skalibrowany.
- `protocol`: `MatchProtocol` – protokół meczu wykorzystywany w analizie.
- `source`: `VideoSource` – źródło obrazu wideo.

Metody:

- `loadMatchData(protocol: MatchProtocol)` – wczytuje dane meczu z protokołu.
- `attachVideoSource(source: VideoSource)` – podłącza źródło wideo do systemu.
- `initializeAI()` – inicjalizuje moduł sztucznej inteligencji.
- `handleStartButton()` – obsługuje rozpoczęcie analizy przez użytkownika.
- `getProtocoleDeatils()` – pobiera szczegóły aktualnie wczytanego protokołu.

Club

Klasa reprezentująca klub sportowy.

Atrybuty:

- `club_name`: `String` – nazwa klubu.

Metody:

- `clubName()`: `String` – zwraca nazwę klubu.

Coach

Klasa reprezentująca trenera powiązanego z klubem lub drużyną.

Atrybuty:

- coachId: int – identyfikator trenera.
- firstName: int – imię trenera.
- lastName: int – nazwisko trenera.
- role: string – rola trenera w zespole.
- employmentDate: Date – data zatrudnienia trenera.
- club: Club – klub, z którym trener jest powiązany.

Metody:

- club(): Club – zwraca klub przypisany do trenera.

DetectedBall

Klasa reprezentująca piłkę wykrytą na obrazie przez system AI.

Atrybuty:

- cords: PitchCoords – współrzędne piłki na boisku.

Metody:

- getCoords() – zwraca aktualne współrzędne piłki.

DetectedPlayer

Klasa reprezentująca zawodnika wykrytego na obrazie przez system AI.

Atrybuty:

- detectedNumber: int – numer zawodnika wykryty na podstawie obrazu.
- cords: PitchCoords – współrzędne zawodnika na boisku.

Metody:

- linkWithProtocolPlayer(z: Player) – wiąże wykrytego zawodnika z zawodnikiem zapisanym w protokole meczu.
- getCoords() – zwraca aktualne współrzędne zawodnika.

DetectedReferee

Klasa reprezentująca sędziego wykrytego na obrazie przez system AI.

Atrybuty:

- cords: PitchCoords – współrzędne sędziego na boisku.

Metody:

- getCoords() – zwraca aktualne współrzędne sędziego.

Frame

Klasa reprezentująca pojedynczą klatkę obrazu pobraną ze źródła wideo.

Atrybuty:

- id: int – identyfikator klatki.
- hour: int – godzina znacznika czasu.
- minute: int – minuta znacznika czasu.
- second: int – sekunda znacznika czasu.
- resolutionX: int – szerokość obrazu w pikselach.
- resolutionY: int – wysokość obrazu w pikselach.
- map: bitmap – bitmapa przechowująca obraz klatki.

Metody:

- Brak metod wskazanych na diagramie.

IActionConsequence

Interfejs reprezentujący konsekwencję akcji meczowej, na przykład zmianę punktacji lub statystyki.

Atrybuty:

- Brak atrybutów wskazanych na diagramie.

Metody:

- Perform() – wykonuje daną konsekwencję.
- Undo() – cofa wcześniej wykonaną konsekwencję.

IDetectedObject

Interfejs opisujący wspólne cechy obiektów wykrywanych na obrazie.

Atrybuty:

- `boundingBox`: `Rectangle` – prostokąt otaczający wykryty obiekt.
- `detectionConfidence`: `float` – poziom pewności detekcji.

Metody:

- `getCenter()`: `Point` – zwraca środek wykrytego obiektu.

IntStatistic

Klasa reprezentująca statystykę przechowywaną jako liczba całkowita.

Atrybuty:

- `value`: `int` – aktualna wartość statystyki.

Metody:

- `IntStatistic(type, scope, startValue)` – konstruktor statystyki całkowitej.
- `ChangeValue(toAdd: int)` – zmienia wartość statystyki o podaną liczbę.
- `SetValue(value: int)` – ustawia wartość statystyki.
- `GetValue()`: `int` – zwraca wartość statystyki.
- `GetValueAsString()`: `string` – zwraca wartość statystyki jako tekst.

MatchAction

Klasa reprezentująca pojedynczą akcję zawodnika w trakcie meczu.

Atrybuty:

- type: MatchActionType – typ akcji.
- result: MatchActionResult – rezultat akcji.
- player: PlayerInMatch – zawodnik wykonujący akcję.
- actionPositioning: PositioningOnThePitch – ustawienie zawodników i piłki w momencie akcji.
- time: Time – czas wykonania akcji.
- consequences: IList<ActionConsequence> – lista konsekwencji wynikających z akcji.

Metody:

- MatchAction(type, result, player, positioning, time) – konstruktor akcji meczowej.
- AddConsequence(consequence) – dodaje konsekwencję do akcji.
- RemoveConsequence(consequence) – usuwa konsekwencję z akcji.
- UpdateStatistics() – aktualizuje statystyki na podstawie akcji.
- UndoUpdate() – cofa aktualizację statystyk wykonaną przez akcję.

MatchActionResult

Typ wyliczeniowy określający rezultat akcji meczowej.

Wartości:

- Good – akcja poprawna.
- Ace – akcja zakończona zdobyciem punktu.
- Error – akcja błędna.

MatchActionType

Typ wyliczeniowy określający rodzaj akcji siatkarskiej.

Wartości:

- Attack – atak.
- Block – blok.
- Dig – obrona.
- Receptio – przyjęcie; w diagramie nazwa prawdopodobnie oznacza Reception.
- Set – wystawa.
- Serve – zagrywka.

MatchAnalysis

Klasa przechowująca aktualny stan analizy meczu, informacje o drużynach, wymianach, statystykach oraz ustawieniu na boisku.

Atrybuty:

- team1: TeamInMatch – pierwsza drużyna biorąca udział w meczu.
- team2: TeamInMatch – druga drużyna biorąca udział w meczu.
- currentSet: int – aktualny set.
- currentRally: int – aktualna wymiana.
- rallies: IList<MatchRally> – lista wymian rozegranych w meczu.
- matchStatistics: StatisticsSet – statystyki całego meczu.
- positioning: PositioningOnThePitch – aktualne ustawienie zawodników i piłki.

Metody:

- AddAction(action: MatchAction) – dodaje akcję do analizy meczu.
- UndoLastAction(): MatchAction – cofa ostatnią zarejestrowaną akcję.
- GetRallyActions(num): IList<MatchAction> – zwraca akcje przypisane do wybranej wymiany.
- CurrentRally(): int – zwraca numer aktualnej wymiany.
- Score(teamNum: int, points: int) – zmienia wynik wskazanej drużyny.
- UpdatePositioning(positioning): void – aktualizuje ustawienie zawodników i piłki.
- GetTeam1(): TeamInMatch – zwraca pierwszą drużynę.
- GetTeam2(): TeamInMatch – zwraca drugą drużynę.

- GetMatchStatistics(): StatisticsSet – zwraca statystyki meczu.

MatchProtocol

Klasa reprezentująca protokół meczu, czyli zestaw formalnych danych potrzebnych do rozpoczęcia analizy.

Atrybuty:

- matchId: int – identyfikator meczu.
- date: Date – data meczu.
- arena: String – miejsce rozegrania meczu.
- hosts: team – drużyna gospodarzy.
- guests: team – drużyna gości.
- referees: List<MatchReferee> – lista sędziów przypisanych do meczu.

Metody:

- Brak metod wskazanych na diagramie.

MatchRally

Klasa reprezentująca pojedynczą wymianę w meczu.

Atrybuty:

- rallyNumber: int – numer wymiany.
- actions: IList<MatchAction> – lista akcji wykonanych w ramach wymiany.

Metody:

- MatchRally(number: int) – konstruktor wymiany.
- GetNumber(): int – zwraca numer wymiany.
- GetRallyActions() – zwraca akcje należące do wymiany.

MatchReferee

Klasa reprezentująca przypisanie sędziego do meczu wraz z jego rolą.

Atrybuty:

- refereeId: int – identyfikator sędziego.
- role: string – rola sędziego w meczu.

Metody:

- Brak metod wskazanych na diagramie.

MatchStatisticsModule

Klasa odpowiedzialna za rejestrowanie danych statystycznych i przekładanie zdarzeń meczowych na statystyki.

Atrybuty:

- isTrackingActive: boolean – informacja, czy śledzenie zdarzeń jest aktywne.
- analysis: MatchAnalysis – obiekt analizy meczu, do którego trafiają rejestrowane zdarzenia.

Metody:

- startDataRegistration() / StartDataRegistration() – rozpoczyna rejestrację danych meczowych.
- recordVolleyballEvent() / RecordVolleyballEvent() – rejestruje zdarzenie siatkarskie rozpoznane podczas analizy.

NetworkStream

Klasa reprezentująca sieciowe źródło transmisji wideo.

Atrybuty:

- protocol: String – protokół transmisji, np. RTSP, RTMP lub inny protokół strumieniowy.

Metody:

- verifyConnection() – sprawdza poprawność i dostępność połączenia sieciowego.

NotificationService

Klasa odpowiedzialna za wysyłanie powiadomień do użytkowników systemu.

Atrybuty:

- emailGateway: EmailGateway – komponent służący do wysyłania wiadomości e-mail.
- templateEngine: Template – mechanizm generowania treści wiadomości na podstawie szablonów.

Metody:

- notifyReferee(referee, match) – wysła powiadomienie do sędziego o przypisaniu do meczu.
- sendEmail(to, subject, body) – wysła wiadomość e-mail.
- sendSms(phone, msg): void – wysła wiadomość SMS.
- logNotification(event): void – zapisuje informację o wysłanym powiadomieniu.

PercentageStatistic

Klasa reprezentująca statystykę procentową.

Atrybuty:

- value: decimal – aktualna wartość statystyki procentowej.

Metody:

- PercentageStatistic(type, scope, startValue) – konstruktor statystyki procentowej.
- SetValue(value: decimal) – ustawia wartość statystyki.
- GetValue(): decimal – zwraca wartość statystyki.
- GetValueAsString(): string – zwraca wartość statystyki jako tekst.

PitchCoords

Struktura reprezentująca współrzędne położenia na boisku.

Atrybuty:

- x: decimal – współrzędna X.
- y: decimal – współrzędna Y.

Metody:

- Brak metod wskazanych na diagramie.

Player

Klasa reprezentująca zawodnika oraz jego podstawowe dane i statystyki.

Atrybuty:

- playerId: int – identyfikator zawodnika.
- firstName: String – imię zawodnika.
- lastName: String – nazwisko zawodnika.
- jerseyNumber: int – numer na koszulce.
- playerStats: StatisticsSet – zestaw statystyk zawodnika.
- position: String – pozycja zawodnika na boisku.
- captain: bool – informacja, czy zawodnik jest kapitanem.

Metody:

- GetId() – zwraca identyfikator zawodnika.
- GetName() – zwraca imię i nazwisko lub nazwę zawodnika.
- GetJerseyNumber() – zwraca numer zawodnika na koszulce.
- GetPlayerStatistics() – zwraca statystyki zawodnika.

PositioningOnThePitch

Struktura reprezentująca ustawienie zawodników obu drużyn oraz położenie piłki na boisku.

Atrybuty:

- team1Players: PlayerInMatch[6] – zawodnicy pierwszej drużyny znajdujący się na boisku.
- team1PlayerCoords: PitchCoords[6] – współrzędne zawodników pierwszej drużyny.
- team2Players: PlayerInMatch[6] – zawodnicy drugiej drużyny znajdujący się na boisku.
- team2PlayerCoords: PitchCoords[6] – współrzędne zawodników drugiej drużyny.
- ballCoords: PitchCoords – współrzędne piłki.

Metody:

- Brak metod wskazanych na diagramie.

Referee

Klasa reprezentująca sędziego sportowego.

Atrybuty:

- id: Long – identyfikator sędziego.
- firstName: String – imię sędziego.
- lastName: String – nazwisko sędziego.
- refereeClass: RefereeClass – klasa lub poziom uprawnień sędziego.
- email: String – adres e-mail sędziego.
- license: License – licencja sędziowska.
- licenseExpiry: Date – data ważności licencji.

Metody:

- isAvailable(date): bool – sprawdza, czy sędzia jest dostępny w podanym terminie.
- isLicenseValid(): bool – sprawdza, czy licencja sędziego jest ważna.

RefereeRepository

Klasa odpowiedzialna za dostęp do danych sędziów zapisanych w bazie danych.

Atrybuty:

- dataSource: DataSource – źródło danych.
- refereeMapper: RefereeMapper – komponent mapujący dane z bazy na obiekty sędziów.

Metody:

- findAvailable(date): List – wyszukuje sędziów dostępnych w danym terminie.
- findById(id): Referee – pobiera sędziego na podstawie identyfikatora.
- save(referee): void – zapisuje dane sędziego.
- existsConflict(id, date): bool – sprawdza, czy sędzia ma konflikt terminów.

ScoreConsequence

Klasa reprezentująca konsekwencję akcji polegającą na zmianie wyniku meczu.

Atrybuty:

- teamId: int – identyfikator drużyny, której wynik ma zostać zmieniony.
- pointsToAdd: int – liczba punktów do dodania.
- match: MatchAnalysis – analiza meczu, w której zostanie zmieniona punktacja.

Metody:

- Perform() – dodaje punkty wskazanej drużynie.
- Undo() – cofa zmianę punktacji.

Statistic

Klasa abstrakcyjna reprezentująca ogólną statystykę.

Atrybuty:

- type: StatisticType – typ statystyki.
- scope: StatisticScope – zakres statystyki.

Metody:

- Statistic(type, scope) – konstruktor statystyki.
- GetType(): StatisticType – zwraca typ statystyki.
- GetScope(): StatisticScope – zwraca zakres statystyki.
- GetValueAsString(): string – zwraca wartość statystyki w postaci tekstowej.

StatisticConsequence

Klasa reprezentująca konsekwencję akcji polegającą na zmianie statystyk meczowych.

Atrybuty:

- type: MatchActionType – typ akcji, która wywołała konsekwencję.
- result: MatchActionResult – rezultat akcji.
- match: MatchAnalysis – analiza meczu, której statystyki mają zostać zmienione.

Metody:

- Perform() – aktualizuje statystyki na podstawie akcji.
- Undo() – cofa zmiany w statystykach.

StatisticScope

Typ wyliczeniowy określający zakres statystyki.

Wartości:

- Match – statystyka dotycząca całego meczu.
- TeamDuringMatch – statystyka drużyny w konkretnym meczu.
- PlayerDuringMatch – statystyka zawodnika w konkretnym meczu.
- TeamGlobal – statystyka drużyny w ujęciu ogólnym.
- PlayerGlobal – statystyka zawodnika w ujęciu ogólnym.

StatisticsCorrectionView

Klasa reprezentująca widok przeznaczony do korekty danych statystycznych.

Atrybuty:

- analysis: MatchAnalysis – analiza meczu, której dane mogą być korygowane.

Metody:

- Brak metod wskazanych na diagramie.

StatisticsSet

Klasa reprezentująca zestaw statystyk.

Atrybuty:

- integerStatistics: IList<IntStatistic> – lista statystyk całkowitoliczbowych.
- rateStatistics: IList<PercentageStatistic> – lista statystyk procentowych.
- Pts: int – liczba punktów.
- Att: int – liczba ataków.
- Succ: int – liczba skutecznych akcji.
- Aces: int – liczba asów serwisowych.
- Blk: int – liczba bloków.
- Digs: int – liczba obron.

Metody:

- StatisticsSet(scope: StatisticScope) – konstruktor zestawu statystyk.
- AddAttackValue(value: int) – dodaje wartość do liczby ataków.
- AddSuccessfulAttackValue(value: int) – dodaje wartość do liczby skutecznych ataków.
- AddBlockValue(value: int) – dodaje wartość do liczby bloków.
- AddDigValue(value: int) – dodaje wartość do liczby obron.
- AddAceValue(value: int) – dodaje wartość do liczby asów.
- AddReceptionValue(value: int) – dodaje wartość do liczby przyjęć.

StatisticType

Typ wyliczeniowy określający rodzaj statystyki.

Wartości:

- NumAttacks – liczba ataków.
- NumBlocks – liczba bloków.
- NumDigs – liczba obron.
- NumAces – liczba asów.
- NumErrors – liczba błędów.
- AttackSuccessRate – skuteczność ataku.
- ReceptionRate – skuteczność przyjęcia.

Team

Klasa reprezentująca drużynę sportową.

Atrybuty:

- teamId: Int – identyfikator drużyny.
- nazwa: String – nazwa drużyny.
- staff: List<Coach> – lista trenerów lub członków sztabu.
- players: List<Player> – lista zawodników.
- teamStats: StatisticsSet – statystyki drużyny.
- average_number_of_points: int – średnia liczba punktów zdobywanych przez drużynę.

Metody:

- GetId() – zwraca identyfikator drużyny.
- GetPlayers() – zwraca listę zawodników.
- GetTeamStatistics() – zwraca statystyki drużyny.
- Captain(): Player – zwraca kapitana drużyny.
- PercentWinRate(): float – zwraca procent zwycięstw drużyny.
- Record(): Tuple<int,int> – zwraca rekord drużyny, prawdopodobnie jako para wartości liczbowych.

User

Klasa reprezentująca użytkownika systemu.

Atrybuty:

- id: Int – identyfikator użytkownika.
- login: String – login użytkownika.
- role: String – rola użytkownika w systemie.

Metody:

- autoryzuj(User) – autoryzuje użytkownika i sprawdza jego uprawnienia.

VideoFile

Klasa reprezentująca źródło wideo w postaci pliku.

Atrybuty:

- filePath: String – ścieżka do pliku wideo.

Metody:

- checkIntegrity() – sprawdza integralność i poprawność pliku wideo.

VideoSource

Klasa abstrakcyjna reprezentująca ogólne źródło obrazu wideo.

Atrybuty:

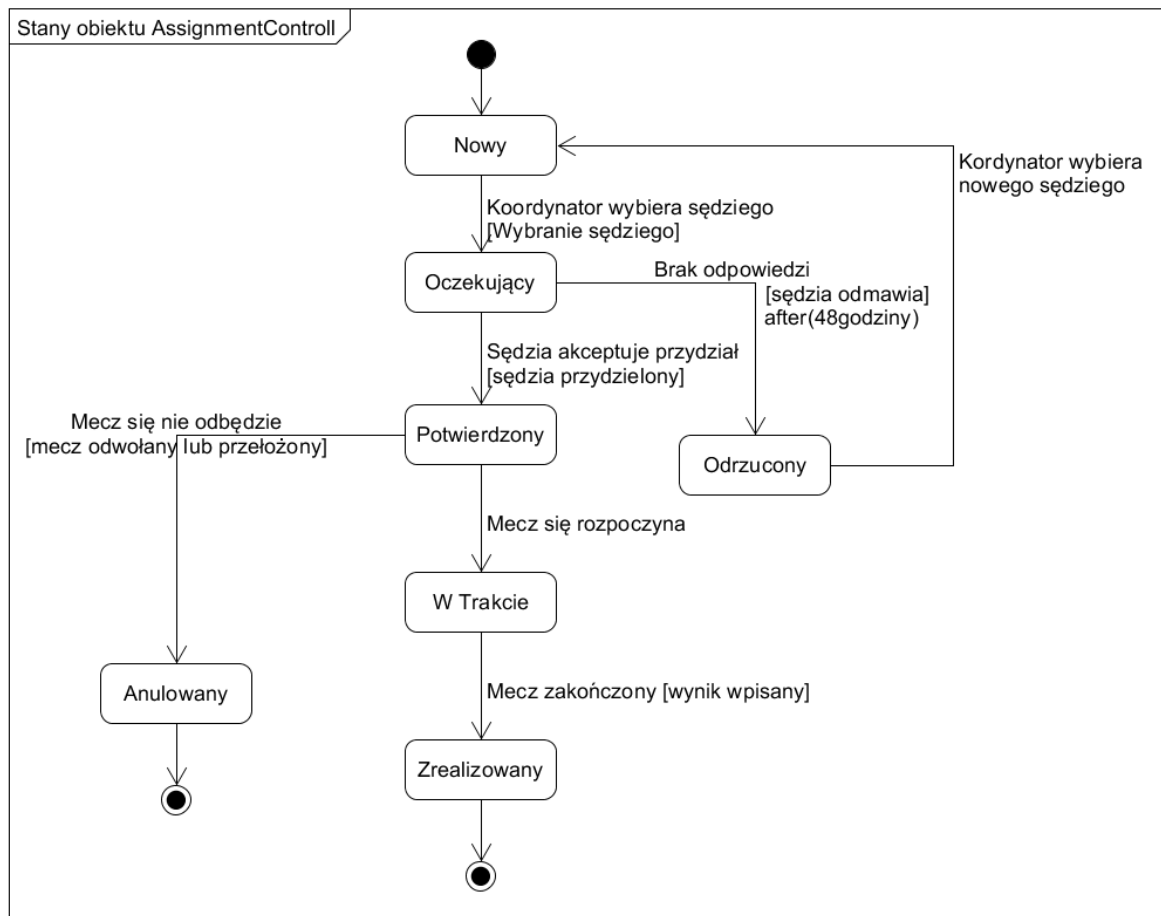
- sourceId: String – identyfikator źródła wideo.
- url: String – adres lub ścieżka źródła.
- isActive: boolean – informacja, czy źródło jest aktywne.
- klatki: List<Frame> – lista klatek pobranych ze źródła.

Metody:

- openStream() – otwiera strumień wideo.
- wyslijKlatke(): Frame – wysyła lub zwraca pojedynczą klatkę obrazu.

Diagramy stanów

Mateusz Tręda



Rysunek 11 Diagram stanów dla obiektu AssignmentControll

Stan: NOWY

stan początkowy tworzony automatycznie przy planowaniu meczu. Brak jeszcze przydzielonego sędziego. System wstępnie pobiera listę dostępnych kandydatów. Przejście wychodzące: koordynator wybiera sędziego → OCZEKUJĄCY.

Stan: OCZEKUJĄCY

przydział zaproponowany, system wysyła powiadomienie e-mail/SMS do sędziego. Uruchamia się licznik 48 godzin. Dwa możliwe przejścia: sędzia akceptuje → POTWIERDZONY; sędzia odmawia lub mija timeout → ODRZUCONY.

Stan: POTWIERDZONY

sędzia potwierdził gotowość, obsada jest kompletna. System wysyła potwierdzenie koordynatorowi. Możliwe przejścia: mecz się rozpoczyna → W_TRAKCIE; mecz zostaje odwołany/przełożony → ANULOWANY

Stan: W_TRAKCIE

mecz aktualnie trwa, sędzia jest na boisku. System rejestruje czas startu. Jedyne przejście wychodzące: mecz kończy się i wynik jest wpisany → ZREALIZOWANY.

Stan: ZREALIZOWANY

stan końcowy powodzenia. Przydział zakończony. System aktualizuje historię i statystyki sędziego. Brak przejść wychodzących.

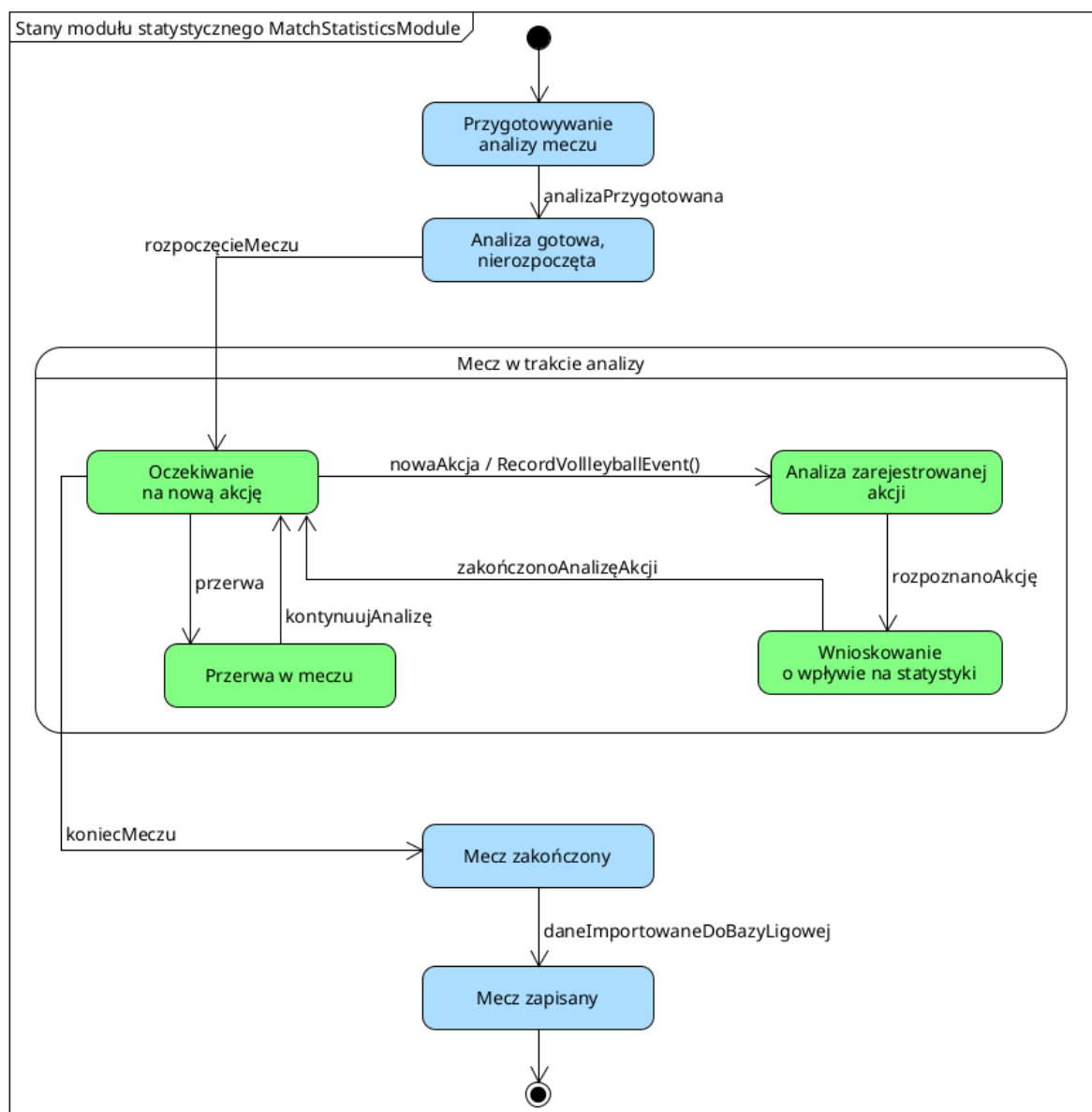
Stan: ODRZUCONY

sędzia odmówił lub nie odpowiedział w ciągu 48h. System generuje alert dla koordynatora. Nie jest to stan końcowy — koordynator wraca do stanu NOWY (linia przerywana), aby wybrać innego sędziego.

Stan: ANULOWANY

stan końcowy awarii organizacyjnej. Mecz odwołany lub przełożony decyzją PZPS. System zwalnia sędziego i anuluje wszystkie powiązane rezerwacje.

Dominik Jarzyło



Rysunek 12 Diagram stanów dla klasy MatchStatisticsModule

Stan: Przygotowywanie analizy meczu

Stan początkowy, przyjmowany przez obiekt klasy MatchStatisticsModule po utworzeniu. Nie są jeszcze zarejestrowane źródła, z których będzie pobierane nagranie do analizy.

Stan: Analiza gotowa, nierozpoczęta

Stan przybierany po zarejestrowaniu źródeł nagrań, na podstawie których będzie dokonywana analiza meczu. Obiekt klasy czeka na rozpoczęcie meczu.

Stan: Mecz w trakcie analizy

Stan złożony, w jakim obiekt klasy MatchStatisticsModule znajduje się w trakcie meczu. Prowadzona jest analiza akcji i statystyk.

Stan: Oczekiwanie na nową akcję

Podstan stanu „Mecz w trakcie analizy”. W tym stanie aktywnie monitorowana jest aktywność zawodników na boisku. System oczekuje na wykonanie akcji przez dowolnego gracza, w której wyniku rejestrowane jest nowe zdarzenie boiskowe i moduł statystyczny przechodzi w stan dogłębnej analizy wykonanej przez siatkarza akcji. Wyjście ze stanu może nastąpić również w wyniku zakończenia meczu lub rozpoczęcia przerwy.

Stan: Analiza zarejestrowanej akcji

Podstan stanu „Mecz w trakcie analizy”. System dokładnie analizuje działania zawodnika na podstawie nagrań. Określany jest gracz wykonujący akcję, typ akcji i jej rezultat (np. akcja została wykonana skutecznie lub była błędna).

Stan: Wnioskowanie o wpływie na statystyki

Podstan stanu „Mecz w trakcie analizy”. Na podstawie danych zebranych z analizy AI nagrań z kamer następuje wnioskowanie o wpływie ostatnio zarejestrowanej akcji na statystyki meczu. Określane są konsekwencje akcji, system decyduje o ewentualnym przyznaniu punktów jednej z drużyn i o konieczności zmiany rotacji. Po zakończeniu wnioskowania dane zapisywane są przez obiekt klasy MatchAnalysis i następuje powrót do stanu „Oczekiwanie na nową akcję”.

Stan: Przerwa w meczu

Podstan stanu „Mecz w trakcie analizy”. W tym stanie analiza zdarzeń na boisku jest wstrzymana do momentu zakończenia przerwy.

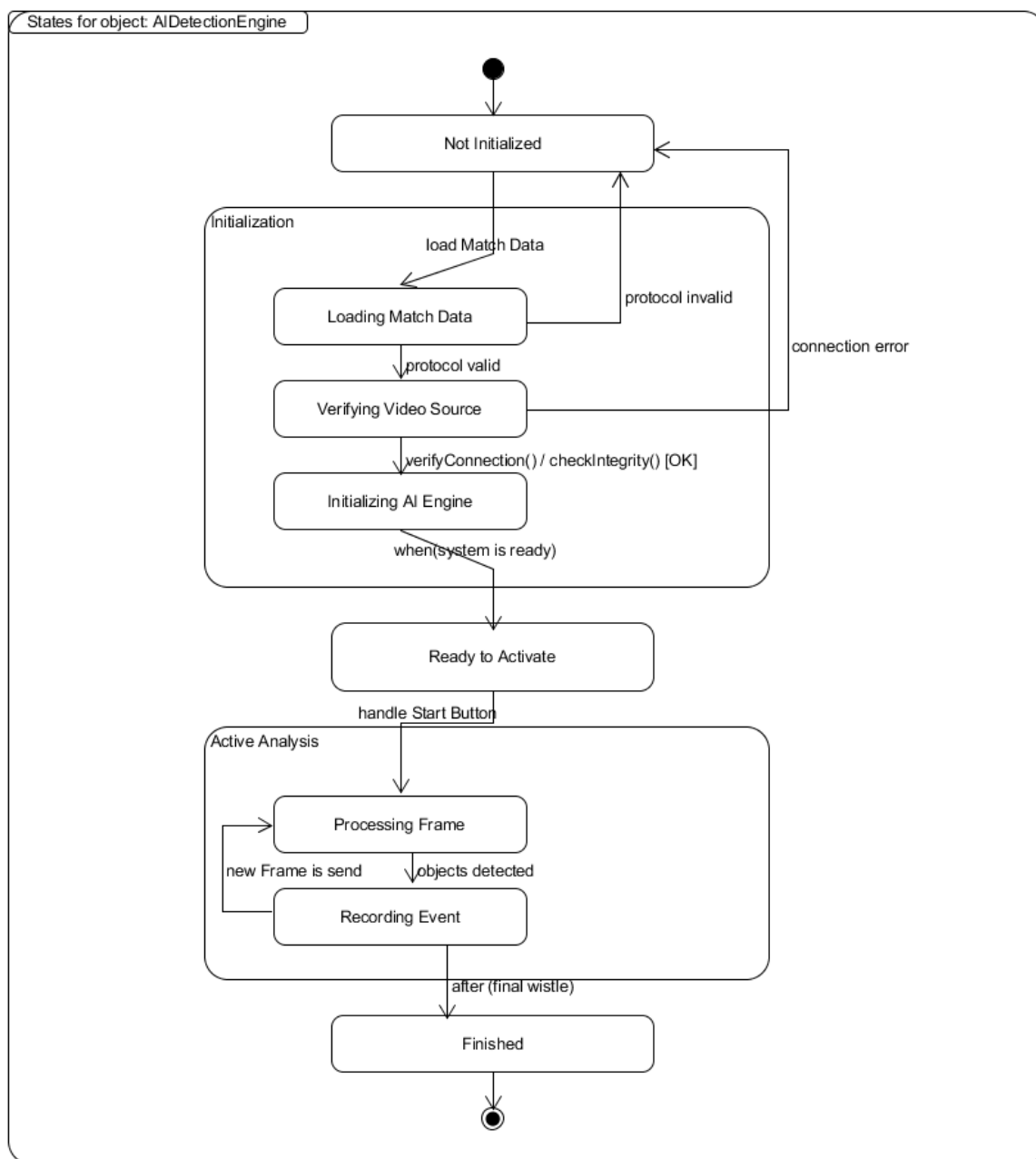
Stan: Mecz zakończony

Automatyczna analiza meczu została zakończona. Jednak dane nie zostały jeszcze zapisane do bazy danych – wciąż możliwa jest ich ewentualna weryfikacja i dokonanie korekty przez statystyka.

Stan: Mecz zapisany

Stan końcowy. Analiza meczu została zakończona, a dane wynikowe zostały zapisane w ligowej bazie danych.

Karol Kuc



Rysunek 13 Diagram stanu dla obiektu AiDetectionEngine

Stan: Not Initialized

Stan początkowy, przyjmowany przez obiekt klasy AiDetectionEngine bezpośrednio po jego utworzeniu. Silnik detekcji nie posiada jeszcze żadnych danych meczowych, żadnego

skonfigurowanego źródła wideo ani załadowanych modeli detekcji. Obiekt oczekuje na dostarczenie protokołu meczowego przez kontroler BroadcastAnalysisController.

Stan: Initialization (stan złożony)

Stan złożony obejmujący cały proces przygotowania systemu do analizy transmisji. Zawiera trzy substany wykonywane sekwencyjnie. Z każdego z substanów możliwy jest powrót do stanu Not Initialized w przypadku wykrycia błędu: niepoprawnego protokołu (protocol invalid) lub błędu połączenia ze źródłem wideo (connection error). Wyjście ze stanu następuje po pomyślnym zakończeniu wszystkich trzech substanów — gdy spełniony jest warunek when(system is ready).

Podstan: Loading Match Data — *podstan stanu „Initialization”*

Obiekt klasy AIDetectionEngine pobiera i przetwarza protokół meczowy (obiekt klasy MatchProtocol) zawierający dane o meczu: identyfikator, datę, nazwę areny, składy drużyn oraz sędziów. Jeżeli protokół jest poprawny (protocol valid), system przechodzi do substanu Verifying Video Source. W przypadku nieprawidłowych lub niekompletnych danych (protocol invalid) następuje wyjście ze stanu złożonego i powrót do stanu Not Initialized.

Podstan: Verifying Video Source — *podstan stanu „Initialization”*

System weryfikuje dostępność i spójność źródła wideo. W zależności od jego typu wywoływana jest metoda verifyConnection() dla strumienia sieciowego (NetworkStream) lub checkIntegrity() dla pliku wideo (VideoFile). Pomyślna weryfikacja powoduje przejście do substanu Initializing AI Engine. Błąd połączenia (connection error) powoduje wyjście ze stanu złożonego i powrót do stanu Not Initialized.

Podstan: Initializing AI Engine — *podstan stanu „Initialization”*

Silnik detekcji AI jest inicjalizowany i kalibrowany. Pobierana jest pierwsza klatka inicjalizacyjna (wyslijKlatke()), która jest analizowana przez metodę analyzeInitializationFrame(). Następnie wywoływane są metody mapPlayers(), findReferee() oraz findBall(), które odpowiednio: przypisują wykrytych zawodników do rekordów protokołu meczowego, lokalizują sędziego i piłkę na boisku. Po zakończeniu kalibracji spełniony jest warunek when(system is ready) i obiekt opuszcza stan złożony Initialization.

Stan: Ready to Activate

Stan oczekiwania na uruchomienie analizy przez operatora. Silnik detekcji jest w pełni skonfigurowany i skalibrowany — modele są załadowane, zawodnicy zmapowani, a obiekty boiskowe zlokalizowane. Obiekt klasy AIDetectionEngine pozostaje w tym stanie do momentu naciśnięcia przez operatora przycisku START (wywołanie handleStartButton()), co powoduje przejście do stanu złożonego Active Analysis.

Stan: Active Analysis (stan złożony)

Stan złożony reprezentujący aktywną fazę analizy transmisji wideo w trakcie meczu siatkarskiego. System przetwarza kolejne klatki wideo w pętli, na przemian wykrywając obiekty na boisku i rejestrując zdarzenia meczowe. Stan jest opuszczany po usłyszeniu końcowego gwizdka sędziego (after(final whistle)).

Podstan: Processing Frame — *podstan stanu „Active Analysis”*

Pierwszy substan stanu Active Analysis. System pobiera kolejną klatkę wideo ze źródła (wyslijKlatke()) i przetwarza ją za pomocą silnika detekcji, wywołując metody findReferee() oraz findBall(). Po pomyślnym wykryciu obiektów na klatce (objects detected) system przechodzi do substanu Recording Event w celu zarejestrowania zaistniałego zdarzenia siatkarskiego.

Podstan: Recording Event — *podstan stanu „Active Analysis”*

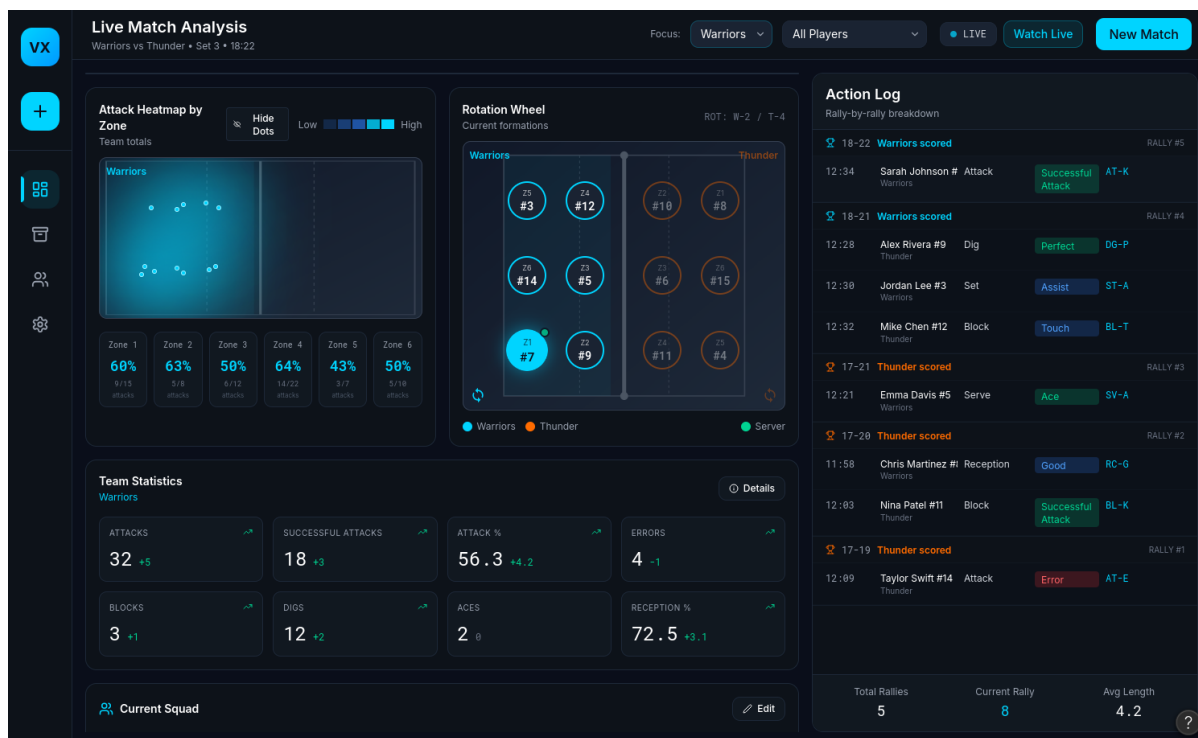
Na podstawie wyników detekcji moduł statystyk meczowych (MatchStatisticsModule) rejestruje zdarzenie siatkarskie poprzez wywołanie metody recordVolleyballEvent(). Rejestrowane mogą być zdarzenia takie jak zagrywka, atak, blok czy błąd. Po zakończeniu rejestracji, gdy dostarczona zostanie nowa klatka wideo (new Frame is send), system powraca do substanu Processing Frame, kontynuując pętlę analizy.

Stan: Finished

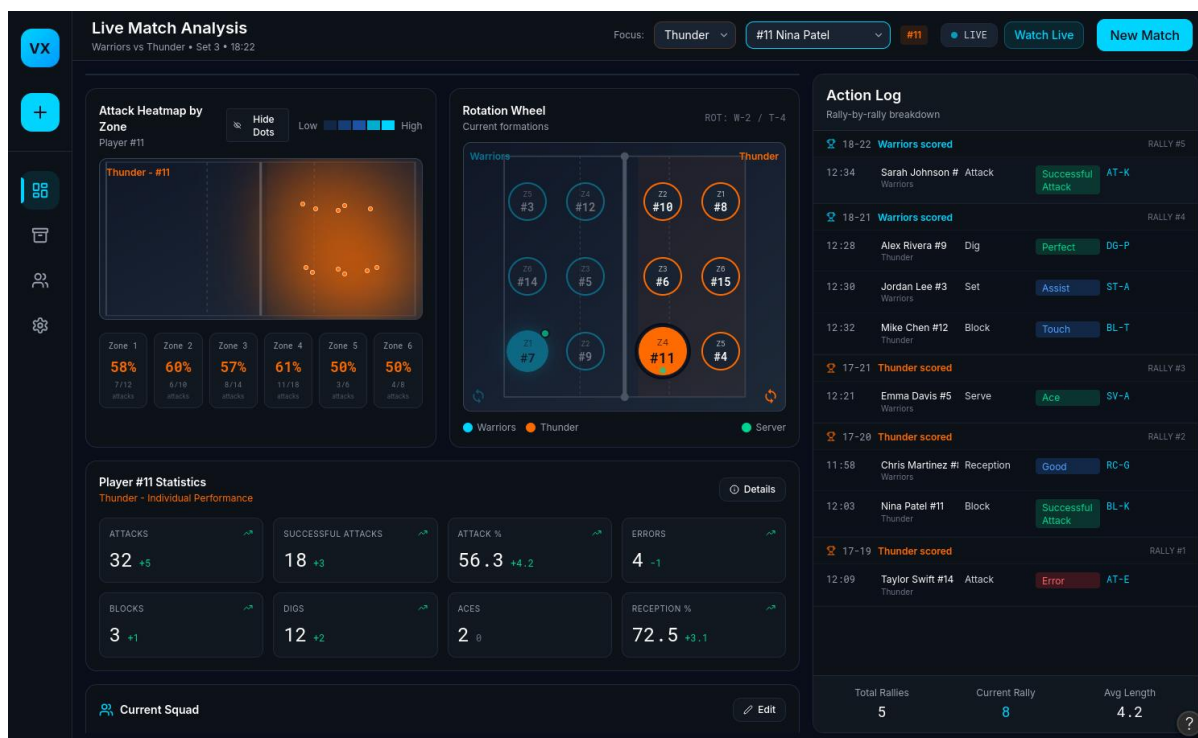
Stan przyjmowany przez obiekt po zakończeniu meczu (sygnał after(final whistle)). Automatyczna analiza transmisji wideo została zakończona — silnik detekcji AI nie przetwarza już kolejnych klatek. Jest to stan końcowy — po jego osiągnięciu obiekt klasy AIDetectionEngine przechodzi bezpośrednio do pseudostanu końcowego, co oznacza zakończenie cyklu życia obiektu.

Propozycje interfejsu użytkownika

Zaproponowano następujący układ menu głównego aplikacji. Wygenerowano go w narzędziu „figma”.



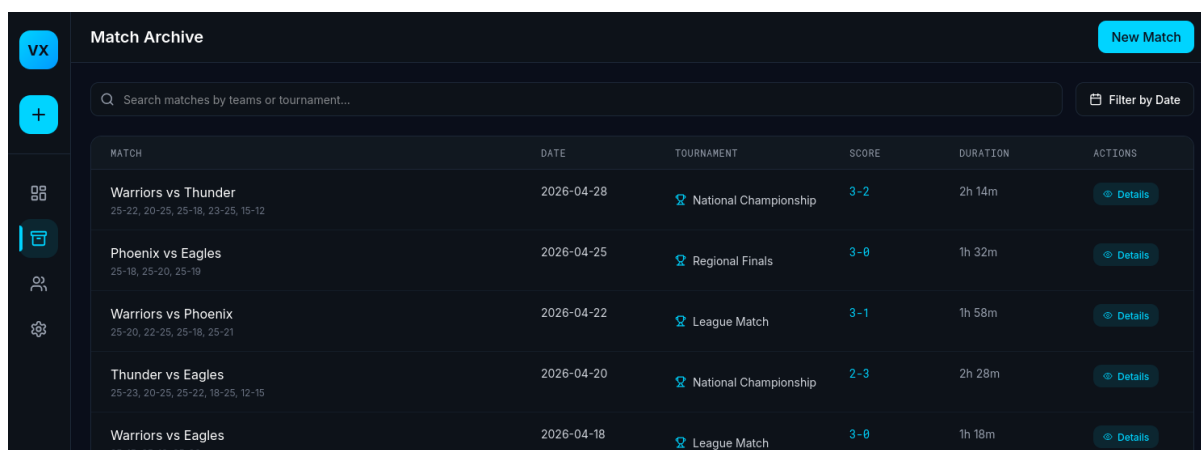
Rysunek 14 Propozycja menu głównego z podglądem analizy meczu



Rysunek 15 Menu główne ze skupieniem na wybranego zawodnika

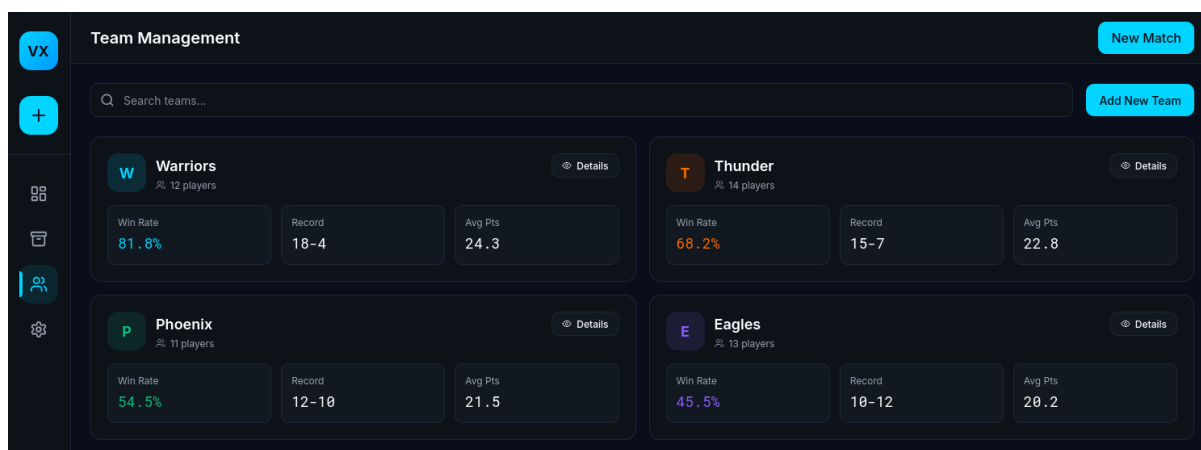
Zakładki umieszczone po lewej stronie okna umożliwiają zmianę widoku na:

- Widok główny analizy,
- Widok archiwum,
- Menu zarządzania drużynami,
- Ustawienia.



MATCH	DATE	TOURNAMENT	SCORE	DURATION	ACTIONS
Warriors vs Thunder 25-22, 20-25, 25-18, 23-25, 15-12	2026-04-28	National Championship	3-2	2h 14m	Details
Phoenix vs Eagles 25-18, 25-20, 25-19	2026-04-25	Regional Finals	3-0	1h 32m	Details
Warriors vs Phoenix 25-20, 22-25, 25-18, 25-21	2026-04-22	League Match	3-1	1h 58m	Details
Thunder vs Eagles 25-23, 20-25, 25-22, 18-25, 12-15	2026-04-20	National Championship	2-3	2h 28m	Details
Warriors vs Eagles 25-18, 25-18, 25-20	2026-04-18	League Match	3-0	1h 18m	Details

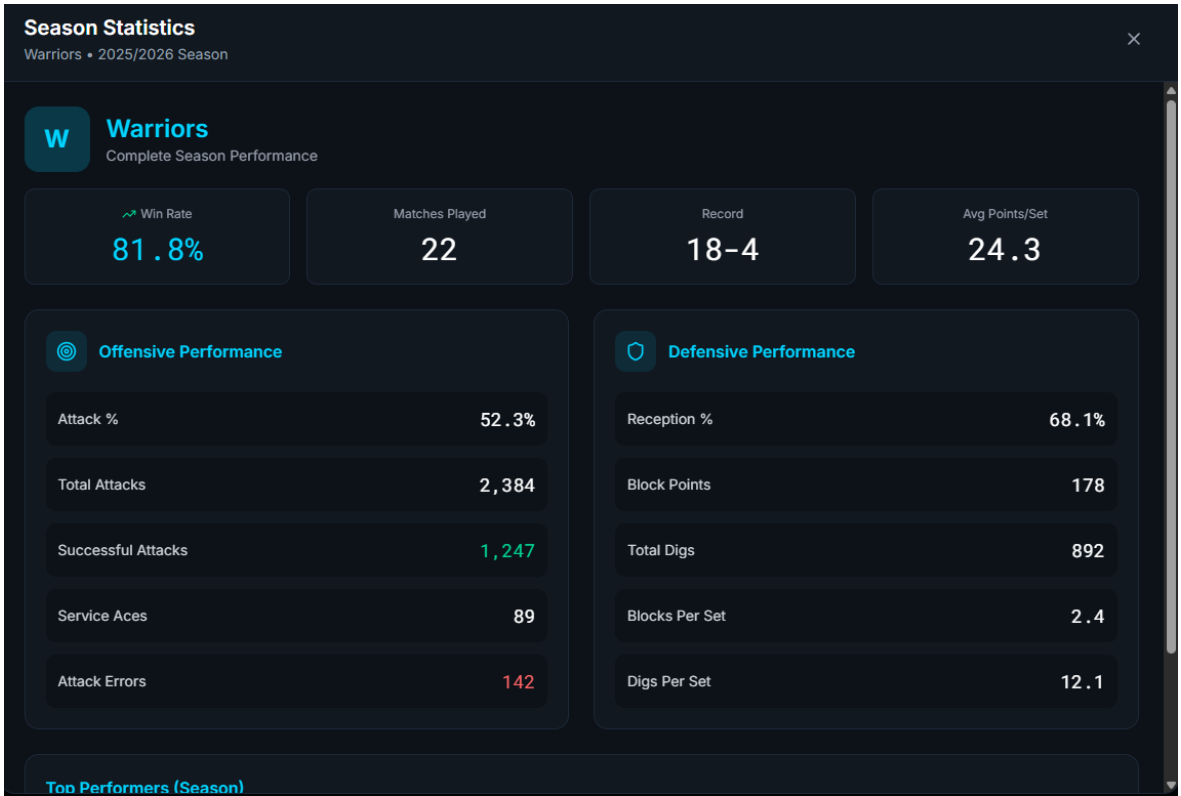
Rysunek 16 Ekran archiwum wcześniejszych meczy



Team	Win Rate	Record	Avg Pts
W Warriors 12 players	81.8%	18-4	24.3
T Thunder 14 players	68.2%	15-7	22.8
P Phoenix 11 players	54.5%	12-10	21.5
E Eagles 13 players	45.5%	10-12	20.2

Rysunek 17 Ekran zarządzania drużynami

W menu zarządzania drużynami możliwy jest również podgląd całosciowych statystyk danego zespołu. Po naciśnięciu przycisku *Details* przy interesującym nas wpisie ukazuje się poniższe okno.



Rysunek 18 Okno szczegółowych statystyk danego zespołu

Naciśnięcie przycisku *New Match* powoduje otwarcie okna przygotowania analizy nowego meczu.

New Match Setup
Step 1 of 3

Video Source

RTSP Stream
Live camera feed

Upload MP4
Recorded match

RTSP Stream URL

rtsp://camera.example.com:554/stream

Enter the RTSP URL for your camera feed

Cancel Continue

Rysunek 19 Ekran przygotowywania nowego meczu - możliwość skonfigurowania analizy obrazu z kamery na żywo

New Match Setup

Step 2 of 3

Team A

Warriors

Team B

Thunder

Tournament / League

e.g., National Championship 2026

Match Date

dd . mm . rrrr

Back

Continue

Rysunek 20 Ekran przygotowywania nowego meczu - wybór zespołów i turnieju

New Match Setup

Step 3 of 3

AI Calibration

Optimize AI models for this specific court and camera angle

☒ Ball tracking

☒ Player detection

☒ Action recognition

☐ Court boundary detection

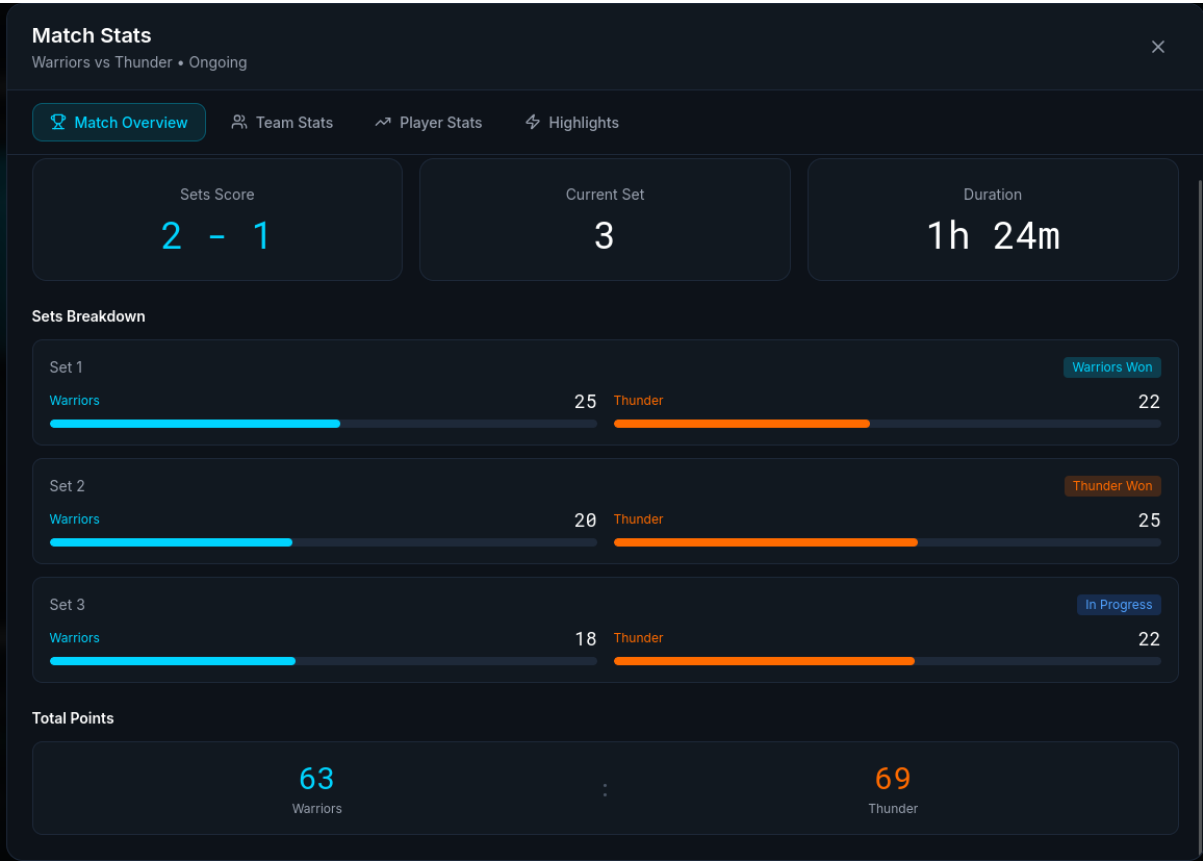
Tip: AI calibration runs automatically in the background. You can start analyzing immediately.

Back

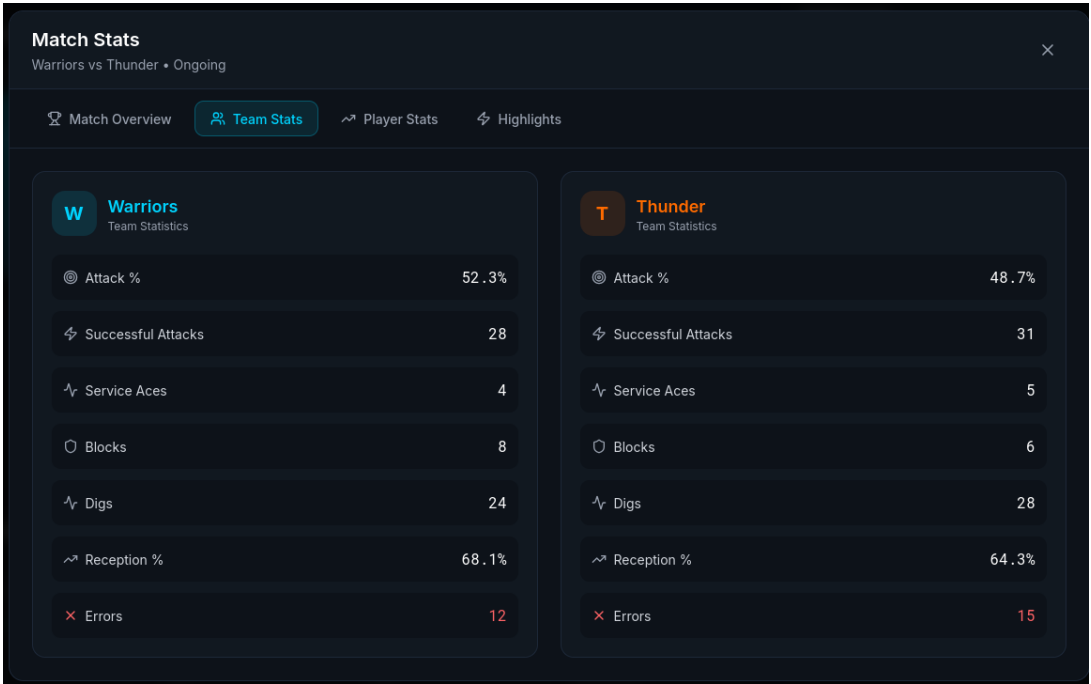
Start Match

Rysunek 21 Ekran przygotowania nowego meczu – opcje kalibracji AI

Po naciśnięciu przycisku *Details* na ekranie głównym przy statystykach zespołu ukazuje się okno ze szczegółowymi statystykami na temat danego meczu, gry drużyny i zawodników podczas aktualnego meczu oraz opis kluczowych wydarzeń w trakcie gry.



Rysunek 22 Okno szczegółowych statystyk na temat danego meczu



Rysunek 23 Okno szczegółowych statystyk obu zespołów w czasie trwania meczu

Match Stats

Warriors vs Thunder • Ongoing

Match Overview

Team Stats

Player Stats

Highlights

Warriors Players

#	PLAYER	POS	PTS	ATT	● SUCC	⬆ ACES	○ BLK	⬆ DIGS
7	Sarah Johnson OH	OH	18	32	18	2	3	12
12	Mike Chen MB	MB	12	20	11	0	4	3
5	Emma Davis S	S	8	15	7	1	0	18
9	Alex Rivera L	L	4	8	3	0	0	22
3	Jordan Lee OH	OH	15	28	14	1	1	8
14	Taylor Swift MB	MB	6	12	5	0	0	4

Thunder Players

#	PLAYER	POS	PTS	ATT	● SUCC	⬆ ACES	○ BLK	⬆ DIGS
8	Chris Martinez OH	OH	16	30	15	1	2	10
11	Nina Patel S	S	10	18	9	2	1	15

Rysunek 24 Okno szczegółowych statystyk poszczególnych zawodników w trakcie meczu

Match Stats

Warriors vs Thunder • Ongoing

Match Overview

Team Stats

Player Stats

Highlights

Match Highlights & Key Moments

⚡

Sarah Johnson #7
Warriors

Scored 5 consecutive aces

12:34

🛡

Maya Kim #4
Thunder

Triple block for the point

11:18

🏐

Jordan Lee #3
Warriors

Amazing dig save leading to score

10:05

🎯

Chris Martinez #8
Thunder

Perfect kill from Zone 4

08:42

Rysunek 25 Przegląd najważniejszych wydarzeń w trakcie meczu

Rejestracja nowej drużyny jest możliwa poprzez formularz dostępny poprzez wybranie przycisku *Add New Team* w menu zarządzania zespołami.

Register New Team

Enter team details and assign players to positions

Team Information

Team Name *

Enter team name

Coach Name (Optional)

Enter coach name or email

Team Logo (Optional)

Upload Logo

Player Roster

+ Add Player

1

Player Name *

Enter name

Position *

Select position

Jersey # *

#

* Required fields

Cancel

Save Team

Rysunek 26 Okno dodawania nowej drużyny

6.) Wymagania нефunkcjonalne

Szacowana wielkość bazy danych przy 100 lig przy 30 000 zarejestrowanych użytkowników:

Na jedną drużynę przypada około:

1. ~21 użytkowników
 - a.) identyfikator = 4B
 - b.) login ~ średnio 15 znaków = $15 * 4B = 60B$
 - c.) rola ~ średnio 12 znaków = $12 * 4B = 48B$

W tym:

2. 20 zawodników
 - a.) numer = 4B
 - b.) Imię, Nazwisko ~ 36 znaków = $36 * 4 = 144B$
 - c.) Kapitan ~ 1bit
 - d.) Statystyki ~ 6 atrybutów * 4B = 12B
3. 1 trener
 - a.) data_zatrudnienia ~8B
 - b.) Klub = referencja do obiektu 4B + 24B obiekt Klub z nazwą klubu = 28B

Razem: ~309B

Wymagany zasób do stworzenia kontrolera API zarządzający systemem:

Przykład (End Point do przydzielania sędziego):

1. AssignmentController ~ 24B
2. Assignment ~ 32B
3. Referee ~70B
4. RefereeRepository ~ 18B
5. NotificationService ~ 34B

Razem: ~178B

$309B * 15 \text{ drużyn} = 4635 \text{ B danych użytkowników}$

Szacowana liczba meczy ~10 000

$10000B * 4635B = 46,35 \text{ MB danych użytkowników w każdym meczu}$

Analiza AI / Computer Vision

Przy czym zbiera się dane takie jak:

- pozycje zawodników
- trajektorie piłki
- wykryte akcje
- embeddingi AI

Przy założeniu, że średni mecz jest odtwarzany w 25 klatkach/s i trwa 2 godziny to wychodzi. 200–500 MB danych analitycznych

Wymagane czasy odpowiedzi

Określamy maksymalny całkowity czas od wysłania żądania przez aplikację użytkownika do wyświetlenia przez nią odpowiedzi. Zakładamy przy tym, że użytkownik posiada łącze o szybkości co najmniej 1 Mb/s i korzysta z serwisu na współczesnym urządzeniu pozwalającym komfortowo przeglądać zasoby internetowe. W przypadku wolniejszego łącza internetowego czas odpowiedzi będzie znacznie wydłużony.

- Utworzenie nowego konta użytkownika ~4.2s
- Pobranie listy sędziów ~ 1.0s
- Operacja przydziału sędziego do meczu ~ 2.1s
- Wysyłanie notyfikacji do sędziego o przydziale ~2.3s
- Potwierdzenie operacji i zapis do bazy danych ~2.7s
- Utworzenie drużyny siatkarskiej ~ 3.2s
- Pobranie wydarzeń podczas meczu ~1.1s
- Pobranie listy zawodników ~1.3s

7.) Technologie Informatyczne

W celu zaimplementowania automatycznej analizy meczu na bazie transmisji wideo z kamer proponowane jest użycie biblioteki widzenia maszynowego OpenCV. Technologia ta pozwala na analizę sceny trójwymiarowej na bazie obrazów dwuwymiarowych oraz wykrywanie obiektów i lokalizację ich w przestrzeni. Na bazie zebranych przy użyciu OpenCV danych możliwe będzie następnie określenie pozycji piłki i poszczególnych zawodników oraz rozpoznawanie wykonywanych przez nich akcji.

Na głównym serwerze systemu uruchamiane będzie oprogramowanie dokonujące właściwej analizy meczu oraz umożliwiające zarządzanie meczami, halami oraz sprzętem.

Serwer dokonujący analizy meczu potrzebuje bardzo dużej mocy obliczeniowej do przeprowadzania jej w czasie rzeczywistym. W związku z tym proponowana jest następująca konfiguracja:

- Procesor Intel Core i9 najnowszej generacji,
- 64 GB pamięci RAM,
- Akcelerator graficzny NVIDIA A100 z 40 GB pamięci VRAM,
- Dysk SSD o pojemności 1 TB,
- Połączenie sieciowe za pomocą światłowodu.

Serwer przechowujący ligową bazę danych wymaga procesora o wysokiej liczbie rdzeni oraz niezawodnej pamięci masowej. Zatem proponowana jest konfiguracja:

- Procesor Intel Xeon 6,
- 32 GB pamięci RAM,
- Macierz dyskowa RAID o pojemności 16 TB.

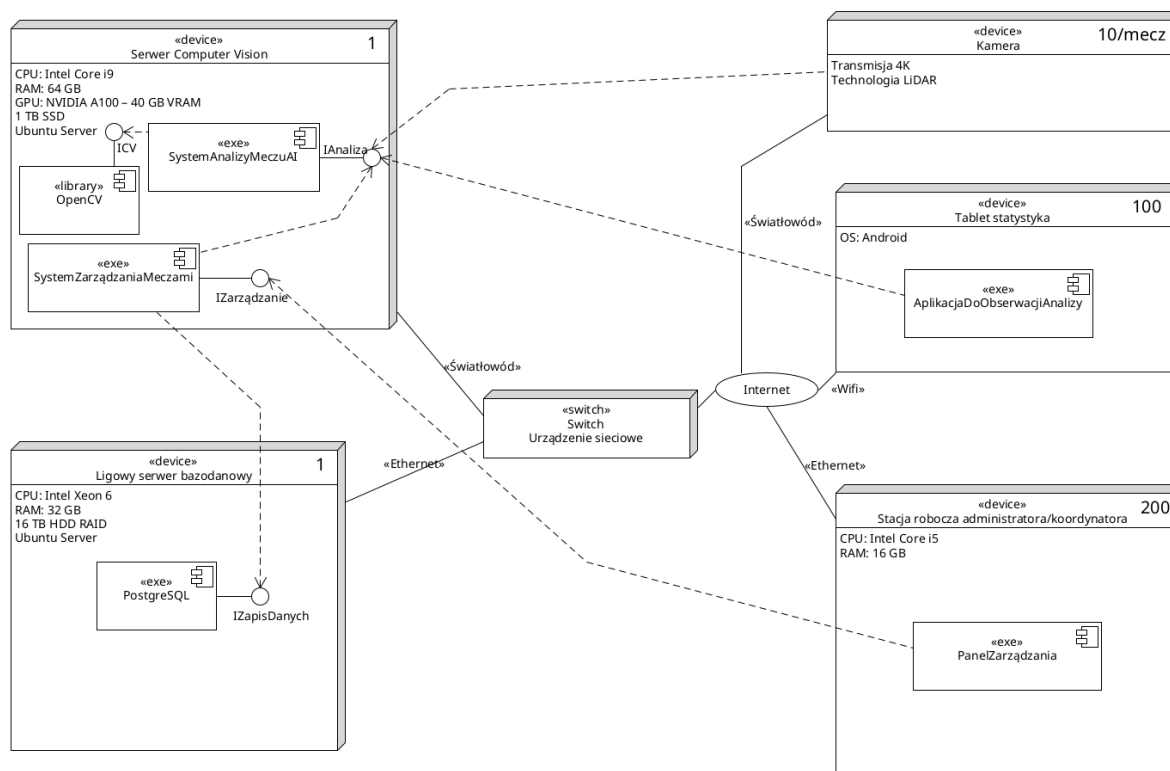
Jako oprogramowanie bazodanowe zostanie wykorzystane PostgreSQL.

Oba serwery połączone są ze sobą za pomocą przełącznika (switch). Bezpośredni dostęp do Internetu będzie miał jedynie serwer systemu analizy.

Poprzez sieć Internetową z systemem łączyć się będą stacje robocze administratorów oraz koordynatorów, z których będą oni wprowadzać nowe dane do systemu, dokonywać rezerwacji hal oraz sprzętu. Statystycy podczas trwania meczu obserwują przeprowadzaną przez system

analizę i dokonują ewentualnej korekty wyznaczonych statystyk za pomocą specjalnej aplikacji na tabletach z systemem Android.

Transmisja z kamer o wysokiej rozdzielczości, w miarę możliwości wyposażonych dodatkowo w system LiDAR, przekazywana będzie do serwera dokonującego analizy za pomocą wysokoprzepustowego połączenia światłowodowego.



Rysunek 27 Proponowany diagram wdrożenia

8.) Analiza ryzyka i zagrożenia

Poniżej przedstawiono listę potencjalnych zagrożeń dotyczących projektu wraz z proponowanymi metodami zapobiegania.

- **Awaria systemu lub problemy z infrastrukturą internetową w trakcie trwania meczu**
 - Prawdopodobieństwo wystąpienia: duże,
 - Stopień szkodliwości: średni,
 - Metoda zapobiegania: load-balancing, użycie zapasowych serwerów,
 - Plan awaryjny: przełączenie się na inny, zapasowy serwer dokonujący analizy. W skrajnych przypadkach konieczne może być kontynuowanie wyznaczania statystyk ręcznie.
- **Częste błędy przy automatycznym wyznaczaniu statystyk**
 - Prawdopodobieństwo wystąpienia: duże,
 - Stopień szkodliwości: wysoki,
 - Metoda zapobiegania: weryfikacja wyliczonych przez algorytm statystyk przez statystyka, poprawa modelu AI poprzez trenowanie na większym zbiorze danych,
 - Plan awaryjny: ręczna korekta odpowiednich wartości przez statystyka. Jeśli problem występować będzie szczególnie często, konieczne może być ponowne dokonanie uczenia modelu AI.
- **Stronniczość systemu AI i faworyzacja określonych zespołów, zawodników lub stylów gry**
 - Prawdopodobieństwo wystąpienia: średnie,
 - Stopień szkodliwości: wysoki,
 - Metoda zapobiegania: weryfikacja wyliczonych przez algorytm statystyk przez statystyka, poprawa modelu AI poprzez trenowanie na większym zbiorze danych,
 - Plan awaryjny: ręczna korekta odpowiednich wartości przez statystyka.
- **Błędy wynikające z niedokładnej kalibracji systemu kamer lub niskiej jakości danych wejściowych**
 - Prawdopodobieństwo wystąpienia: wysokie,
 - Stopień szkodliwości: średni,
 - Metoda zapobiegania: zaimplementowanie automatycznego systemu weryfikującego kompletność konfiguracji systemu kamer przed rozpoczęciem meczu.
 - Plan awaryjny: ewentualna korekta konfiguracji i rozstawienia kamer, w ostateczności konieczne może być kontynuowanie wyznaczania statystyk ręcznie.

- **Nadmierne poleganie na danych z systemu przez analityków, trenerów**
 - Prawdopodobieństwo wystąpienia: średnie,
 - Stopień szkodliwości: średni,
 - Metoda zapobiegania: przeszkolenie odpowiednich osób na temat ograniczeń systemów AI, weryfikowanie zebranych danych przez ludzi,
 - Plan awaryjny: –
- **Wyciek danych osobowych użytkowników systemu lub zawodników**
 - Prawdopodobieństwo wystąpienia: średnie,
 - Stopień szkodliwości: wysoki,
 - Metoda zapobiegania: szyfrowanie wrażliwych danych osobowych, zaimplementowanie kontroli dostępu do odpowiednich danych,
 - Plan awaryjny: poinformowanie osób, których dane wyciekły oraz naprawa luki w systemie.
- **Uzyskanie nieautoryzowanego dostępu lub atak hakerski**
 - Prawdopodobieństwo wystąpienia: średnie,
 - Stopień szkodliwości: wysoki,
 - Metoda zapobiegania: implementacja bezpiecznej autentykacji i kontroli dostępu użytkowników, wymuszenie stosowania silnych haseł przez użytkowników, przeprowadzanie audytów bezpieczeństwa, detekcja włamań,
 - Plan awaryjny: odcięcie skompromitowanej części systemu, wydanie aktualizacji łatającej luki w systemie, poprawa zabezpieczeń.
- **Niezgodność systemu z obowiązującymi przepisami prawnymi (np. RODO)**
 - Prawdopodobieństwo wystąpienia: znikome,
 - Stopień szkodliwości: wysoki,
 - Metoda zapobiegania: weryfikacja zgodności systemu z prawem przed jego wdrożeniem, konsultacje prawne,
 - Plan awaryjny: korekta systemu w celu osiągnięcia zgodności z prawem.

9.) Plan Pracy

Etap	Opis prac	Czas trwania	Zależności	Osoby odpowiedzialne
1. Analiza wymagań i doprecyzowanie zakresu systemu	Zebranie wymagań funkcjonalnych i niefunkcjonalnych, analiza przypadków użycia, określenie ról użytkowników: administrator, koordynator, trener, statystyk, zawodnik, PZPS.	5 dni	Brak	Cały zespół
2. Projekt architektury systemu	Określenie podziału systemu na moduły: panel administracyjny, moduł organizacji meczu, analiza wideo AI, dashboard, baza danych, eksport raportów. Przygotowanie modelu komunikacji między modułami.	5 dni	Etap 1	Cały zespół
3. Projekt bazy danych	Zaprojektowanie tabel dla użytkowników, drużyn, zawodników, meczów, hal, sprzętu, sędziów, statystyk, raportów i nagrań. Przygotowanie relacji oraz ograniczeń integralności.	4 dni	Etap 2	1 osoba + konsultacje zespołu
4. Implementacja modułu użytkowników i uprawnień	Logowanie, role użytkowników, zarządzanie kontami, weryfikacja administratora, przypisywanie uprawnień.	7 dni	Etapy 2–3	1 osoba
5. Implementacja modułu organizacji meczu	Funkcje rezerwacji hali, przydziału sędziów, wypożyczania sprzętu, przydziału drużyn i trenerów, przesuwania lub odwoływania meczu.	10 dni	Etapy 3–4	1–2 osoby

6. Implementacja modułu przygotowania transmisji wideo	Obsługa źródeł wideo, wczytywanie pliku lub strumienia, konfiguracja kamer, sprawdzanie poprawności połączenia, przygotowanie danych meczu do analizy.	8 dni	Etapy 2–4	1 osoba
7. Implementacja modułu Computer Vision / AI	Detekcja zawodników, piłki i elementów boiska, śledzenie obiektów, przygotowanie danych przestrzennych do klasyfikacji zagrań.	15 dni	Etap 6	1–2 osoby
8. Implementacja modułu klasyfikacji akcji i statystyk	Rozpoznawanie akcji: zagrywka, przyjęcie, wystawa, atak, blok, obrona. Aktualizacja statystyk meczu i zawodników, obsługa korekty błędnie rozpoznanych zdarzeń.	10 dni	Etap 7	1–2 osoby
9. Implementacja dashboardu analitycznego	Widok meczu na żywo, lista akcji, statystyki, heatmapy, podgląd szczegółów akcji, możliwość korekty danych przez statystyka.	10 dni	Etapy 5, 8	1 osoba
10. Eksport danych i generowanie raportów	Generowanie raportów PDF/XLS, eksport danych do zewnętrznej bazy ligowej, archiwizacja zakończonych meczów.	6 dni	Etapy 8–9	1 osoba
11. Testy systemu	Testy jednostkowe, integracyjne, testy przypadków użycia, testy wydajności analizy wideo, testy poprawności raportów i uprawnień.	8 dni	Etapy 4–10	Cały zespół

12. Wdrożenie pilotażowe	Uruchomienie systemu w środowisku testowym, przygotowanie przykładowych danych, sprawdzenie działania na próbnej transmisji lub nagraniu meczu.	4 dni	Etap 11	Cały zespół
13. Szkolenie użytkowników i dokumentacja	Przygotowanie krótkiej instrukcji obsługi dla administratora, koordynatora, trenera i statystyka. Przeprowadzenie szkolenia z obsługi systemu.	3 dni	Etap 12	Cały zespół

Alokacja zasobów ludzkich:

Osoba / rola	Zakres odpowiedzialności
Członek zespołu 1	Moduł organizacji meczu, rezerwacje hal, sędziowie, sprzęt, terminarz
Członek zespołu 2	Moduł analizy wideo, Computer Vision, klasyfikacja akcji, przetwarzanie danych meczowych
Członek zespołu 3	Panel użytkownika, dashboard, raporty, eksport danych, dokumentacja
Cały zespół	Analiza wymagań, projekt architektury, testy końcowe, wdrożenie i prezentacja systemu

10.) Kosztorys

Kosztorys obejmuje koszty zaprojektowania, implementacji, testowania, wdrożenia oraz utrzymania systemu. Kwoty mają charakter szacunkowy i zostały podane jako koszt netto.

Wariant podstawowy — system demonstracyjny / akademicki

Wariant obejmuje wykonanie systemu w wersji prototypowej: obsługę użytkowników, organizację meczu, podstawową analizę nagrania, dashboard oraz generowanie raportów.

Element kosztorysu	Szacowany koszt
Analiza wymagań i projekt systemu	4 000 zł
Projekt i implementacja bazy danych	3 500 zł
Moduł użytkowników i uprawnień	5 000 zł
Moduł organizacji meczu	7 000 zł
Moduł przygotowania transmisji / wideo	6 000 zł
Podstawowy moduł AI / Computer Vision	12 000 zł
Dashboard analityczny	8 000 zł
Generowanie raportów PDF/XLS	4 000 zł
Testy systemu	5 000 zł
Dokumentacja techniczna i użytkowa	2 500 zł
Wdrożenie demonstracyjne	3 000 zł
Szkolenie użytkowników	2 000 zł

Łączny koszt wariantu podstawowego: 62 000 zł netto

Wariant rozszerzony — system produkcyjny

Wariant rozszerzony zakłada większą dokładność analizy AI, integrację z zewnętrznymi systemami ligowymi, obsługę wielu kamer, archiwum nagrań oraz stabilniejsze wdrożenie produkcyjne.

Element kosztorysu	Szacowany koszt
Analiza wymagań i projekt techniczny	7 000 zł
Baza danych i archiwum meczów	8 000 zł
Panel administracyjny i zarządzanie rolami	8 000 zł
Moduł organizacji rozgrywek	12 000 zł
Obsługa kamer IP / RTSP / RTMP	10 000 zł
Zaawansowany moduł AI / Computer Vision	35 000 zł
Klasyfikacja zagrań i korekta statystyk	18 000 zł
Dashboard z heatmapami i analizą live	15 000 zł
Eksport PDF/XLS oraz integracja z bazą ligową	8 000 zł
Testy wydajnościowe i bezpieczeństwa	10 000 zł
Wdrożenie produkcyjne	8 000 zł
Szkolenie administratorów, trenerów i statystyków	5 000 zł
Konsultacje powdrożeniowe	6 000 zł

Łączny koszt wariantu rozszerzonego: 150 000 zł netto

Koszty oprogramowania i infrastruktury

Pozycja	Koszt
Serwer aplikacyjny / hosting testowy	300–800 zł miesięcznie
Serwer GPU lub usługa chmurowa do analizy AI	1 500–5 000 zł miesięcznie
Baza danych	0–1 000 zł miesięcznie, zależnie od wybranej technologii
Narzędzia programistyczne	0 zł przy wykorzystaniu narzędzi open source
Certyfikat SSL, domena, kopie zapasowe	300–1 000 zł rocznie
Kamery i sprzęt wideo	2 000–15 000 zł, zależnie od liczby kamer

W wersji akademickiej można założyć użycie darmowych narzędzi programistycznych oraz lokalnego środowiska testowego, co znacząco ogranicza koszty infrastruktury.

Warunki płatności

Proponowany sposób płatności:

Transza	Procent wartości projektu	Moment płatności
I transza	20%	Po zaakceptowaniu wymagań i podpisaniu umowy
II transza	30%	Po wykonaniu projektu architektury, bazy danych i podstawowych modułów systemu
III transza	30%	Po dostarczeniu działającej wersji systemu z modułem analizy i dashboardem
IV transza	20%	Po zakończeniu testów, wdrożeniu i odbiorze końcowym

Sposób odbioru projektu

Odbiór projektu następuje po spełnieniu następujących warunków:

1. Dostarczenie działającej wersji systemu.
2. Przekazanie kodu źródłowego oraz dokumentacji.
3. Pozytywne przejście testów funkcjonalnych dla głównych przypadków użycia.
4. Poprawne wygenerowanie raportu z przykładowego meczu.
5. Poprawne działanie ról użytkowników i uprawnień.
6. Prezentacja działania systemu na przykładowym nagraniu lub transmisji testowej.
7. Usunięcie błędów krytycznych wykrytych podczas odbioru.

11.) Aneksy

Umowa powierzenia przetwarzania danych osobowych (RODO)

Administratorem danych jest Polski Związek Piłki Siatkowej, ul. Puławska 300a, 02-819 Warszawa, NIP: 521-000-00-00, reprezentowany przez Prezesa Zarządu. Procesor zobowiązuje się przetwarzać dane osobowe wyłącznie w celu realizacji umowy głównej – dostarczenia, utrzymania i wsparcia systemu zarządzania kadrą siatkówkową. Na podstawie prawnej **Art. 6 ust. 1 lit. b RODO (umowa)** zbiera się dane osobowe zawodników, sędziów, trenerów oraz administratorów systemu.

Obowiązki Procesora

- Przetwarzać dane wyłącznie na udokumentowane polecenie ADO.
- Zapewnić poufność danych – zobowiązać pracowników do zachowania tajemnicy.
- Stosować odpowiednie środki techniczne i organizacyjne (szyfrowanie TLS 1.3, hasła bcrypt, MFA dla adminów).
- Nie powierzać danych podprocesorom bez uprzedniej pisemnej zgody ADO.
- Umożliwić ADO przeprowadzenie audytów i inspekcji.
- Usunąć lub zwrócić wszystkie dane po zakończeniu umowy (maks. 30 dni).
- Zgłaszać naruszenia ochrony danych do ADO w ciągu 24 godzin od wykrycia.

Środki bezpieczeństwa

W celu zapewnienia najwyższego poziomu bezpieczeństwa, zastosowano środki w celu sprywatyzowania transmisji danych. HTTPS protokół internetowy korzystający z dodatkowego szyfrowania. Nasze witryny są zabezpieczone za pomocą certyfikatów SSL. Baza danych jest dostępna wyłącznie przez backed API szyfrowany przez at-rest(AES-256).

Kopie zapasowe są wykonywane w nocy i przechowywane w osobnej strefie geograficznej.

Wszystkie logi są audytowe, czyli każda operacja CRUD jest rejestrowana i znany jest timestamp oraz IP wykonawcy.

Plan testów akceptacyjnych (UAT)

Celem testów UAT jest weryfikacja, że system spełnia wymagania funkcjonalne i niefunkcjonalne określone w specyfikacji. Testy przeprowadzają reprezentanci PZPS przy udziale Wykonawcy.

Harmonogram testów

Faza	Opis	Czas trwania	Odpowiedzialny
Przygotowanie środowiska	Konfiguracja środowiska testowego, załadowanie danych testowych	2 dni	Wykonawca
Testy funkcjonalne	Weryfikacja wszystkich przypadków użycia wg listy C.2	5 dni	PZPS + Wykonawca
Testy wydajnościowe	Symulacja obciążenia: 200 jednoczesnych użytkowników	1 dzień	Wykonawca
Testy bezpieczeństwa	Skan OWASP, testy penetracyjne (zakres zdefiniowany)	2 dni	Zewnętrzny audytor
Raportowanie i retesty	Analiza wyników, poprawki, ponowne testy	3 dni	Obaj

ID	Przypadek użycia	Scenariusz	Oczekiwany wynik
T-01	Rejestracja sędziego	Dodaj nowego sędziego z pełnymi danymi i aktywną licencją	Profil zapisany, nr ewidencyjny wygenerowany, e-mail powitalny wysłany
T-02	Rejestracja sędziego	Próba rejestracji z brakującym polem obowiązkowym	Komunikat błędu walidacji, brak zapisu
T-03	Przydział sędziego	Przydziel sędziego do meczu z aktywną licencją i brak konfliktu	Przydział zapisany, sędzia powiadomiony e-mailem
T-04	Przydział sędziego	Próba przydziału sędziego z wygasłą licencją	System blokuje przydział, wyświetla ostrzeżenie
T-05	Przydział sędziego	Próba przydziału sędziego zajętego w tym samym terminie	Konflikt terminowy wykryty, błąd 409
T-06	Licencja	Odnów licencję sędziego przed wygaśnięciem	Status licencji: ACTIVE, nowa data ważności zapisana
T-07	Powiadomienie	Licencja wygasa za 25 dni – czy system wysła alert?	E-mail do sędziego i koordynatora wysłany o 8:00
T-08	Raport	Generuj raport licencji wygasających w ciągu 30 dni	PDF wygenerowany poprawnie, dane zgodne z BD
T-09	Logowanie	3 błędne próby logowania	Konto zablokowane na 15 minut, e-mail alertowy wysłany
T-10	Terminarz	Zaplanuj mecz na zajętej hali	Konflikt zasobów wykryty, system sugeruje alternatywne terminy
T-11	Odwołanie meczu	Odwołaj mecz z kompletną obsadą sędziowską	Sędzia powiadomiony, przydział anulowany, hala zwolniona
T-12	Wyszukiwanie	Wyszukaj sędziów wg klasy i regionu	Lista filtrowana poprawnie, czas < 1s

Kryteria zaliczenia UAT

- Minimum 95% scenariuszy zaliczonych (bez błędów krytycznych).
- Brak otwartych błędów o priorytecie KRYTYCZNYM lub WYSOKIM.
- Czasy odpowiedzi zgodne z wymaganiami SLA (Aneks J).

Instrukcja instalacji i konfiguracji środowiska

D.1 Wymagania sprzętowe i systemowe

Komponent	Minimum	Zalecane
Serwer aplikacji (VPS)	4 vCPU, 32 GB RAM, 500 GB SSD	8 vCPU, 64 GB RAM, NVIDIA A100, 1 TB SSD NVMe
Serwer bazy danych	2 vCPU, 16 GB RAM, 4 TB SSD	4 vCPU, 32 GB RAM, 16 TB RAID
System operacyjny	Ubuntu 22.04 LTS (64-bit)	Ubuntu 24.04 LTS (64-bit)
Java (backend)	Java 21 LTS (OpenJDK)	Java 21 LTS (GraalVM native)
Node.js (frontend build)	Node.js 20 LTS	Node.js 22 LTS
Baza danych	PostgreSQL 15	PostgreSQL 16
Cache	Redis 7.0	Redis 7.2 (Sentinel HA)
Serwer WWW	Nginx 1.24	Nginx 1.26

D.2 Kolejność instalacji

1. Provisioning serwera VPS – konfiguracja firewalla (UFW), aktualizacja systemu (apt upgrade).
2. Instalacja PostgreSQL 16 – utworzenie bazy danych szks_prod i użytkownika szks_user z ograniczonymi uprawnieniami.
3. Instalacja Redis 7 – konfiguracja maxmemory-policy allkeysrlu, wyłączenie dostępu z zewnątrz.
4. Instalacja Java 21 (OpenJDK) – ustawienie zmiennej JAVA_HOME.
5. Deploy pliku JAR aplikacji backendowej Spring Boot – konfiguracja jako usługa systemd (szks-backend.service).
6. Konfiguracja pliku application.properties: dane połączenia z BD, klucze JWT, dane SendGrid API.
7. Build frontendu React (npm run build) – skopiowanie artefaktów do /var/www/szks.
8. Konfiguracja Nginx – reverse proxy do portu 8080, SSL/TLS (certbot --nginx), wymuszenie HTTPS.
9. Konfiguracja zadań CRON – skrypt sprawdzania wygasających licencji (codziennie 8:00 UTC).
10. Konfiguracja automatycznych backupów – skrypt pg_dump z wysyłką do S3/Backblaze B2.
11. Konfiguracja monitoringu – Prometheus (eksporter JVM i PostgreSQL) + Grafana Dashboard.
12. Test weryfikacyjny – uruchomienie zestawu health-check endpoints.

Podpisanie

Administrator danych (ADO)	Podmiot przetwarzający
Imię i nazwisko: _____	Imię i nazwisko: _____
Stanowisko: _____	Stanowisko: _____
Data: _____	Data: _____
Podpis: _____	Podpis: _____

Data odbioru	_____
Miejsce odbioru	_____
Przedstawiciel Zamawiającego	_____
Przedstawiciel Wykonawcy	_____